Aprendizado de Máquina com Perceptron e Backpropagation

Sarajane Marques Peres
Clodoaldo A. M. Lima

Bibliografia

- Slides baseados em:

  - Fundamentals of Neural Networks: architecture, algorithms and applications. Laurene Fausett. Prentice-Hall, 1994.



Aprendizado de Máquina

- Aprendizado de máquina é caracterizado pela implementação de aprendizado indutivo.
  - Aprendizado baseado em exemplos.

Algumas informações adicionais sobre aprendizado (usando paradigma simbólico)

Raciocínio Indutivo

Conhece: p(a,b), p(a,d), p(d,e), p(d,g), p(e,f)
Observa: a(a,e) e a(d,f)
Aprende: $p(X,Y) \wedge p(Y,Z) \rightarrow a(X,Z)$

Raciocínio Abdutivo

Conhece: p(a,b), p(a,d), p(d,e), p(d,g), p(e,f), $p(X,Y) \wedge p(Y,Z) \rightarrow a(X,Z)$
Observa: a(a,c)
Explica: p(b,c) ou p(d,c)

Raciocínio Dedutivo

Conhece: p(a,b), p(a,d), p(b,c), p(d,e), p(d,g), p(e,f), $p(X,Y) \wedge p(Y,Z) \rightarrow a(X,Z)$
Conclui: a(a,c), a(a,e), a(a,g) e a(d,f)



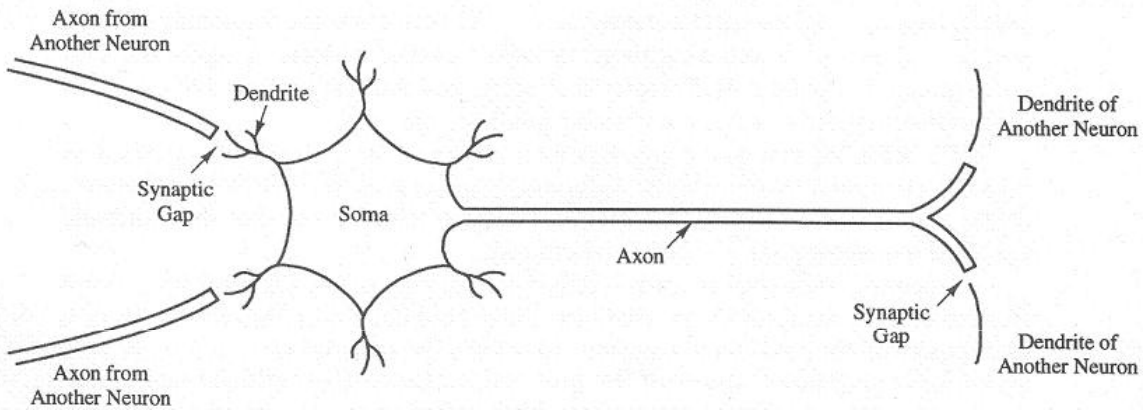
Redes Neurais Artificiais

- É um sistema de processamento de informação que tem algumas características em comum com as redes neurais biológicas.
  - São desenvolvidas como uma generalização de modelos matemáticos da cognição humana ou biologia neural, baseado nos seguintes princípios:
    - O processamento da informação ocorre em vários (muitos) elementos simples chamados neurônios;
    - Sinais são transmitidos entre neurônios por meio de links de conexão;
    - Cada link de conexão tem um peso associado, o qual, em uma rede neural típica, múltiplos sinais são transmitidos;
    - Cada neurônio aplica uma função de ativação (usualmente não linear) em sua entrada para determinar seu sinal de saída.



Preliminares

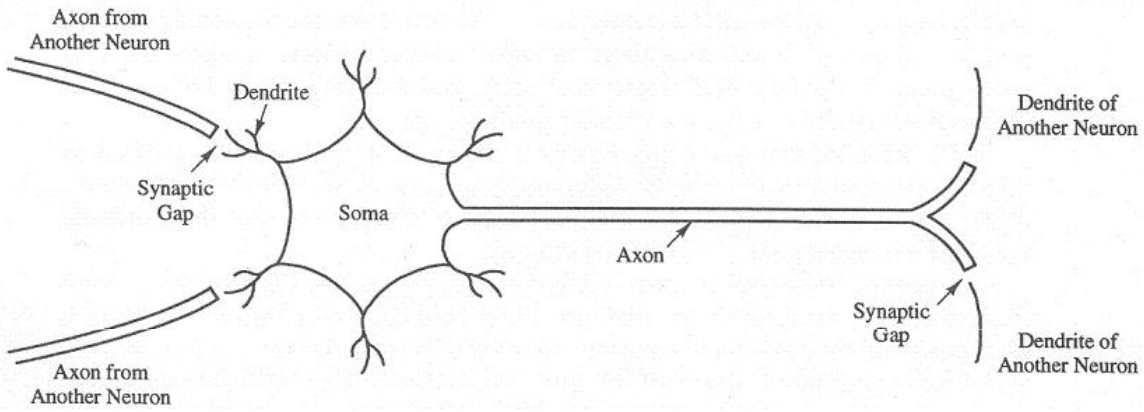
- O neurônio biológico e o neurônio artificial.

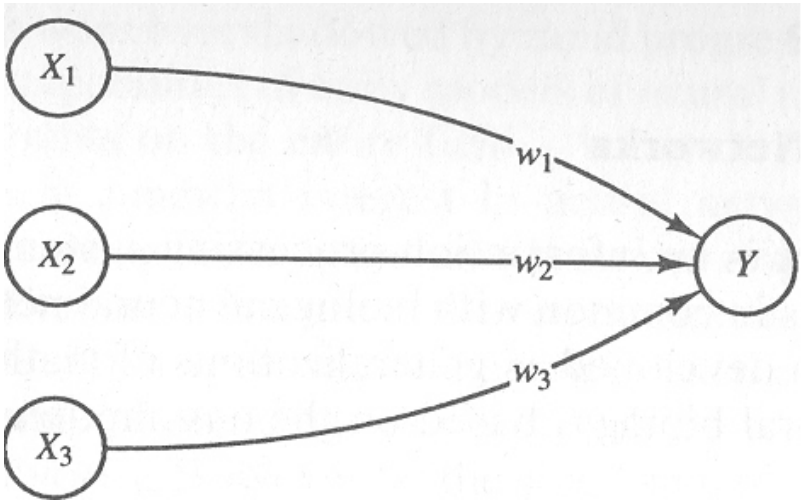




Preliminares

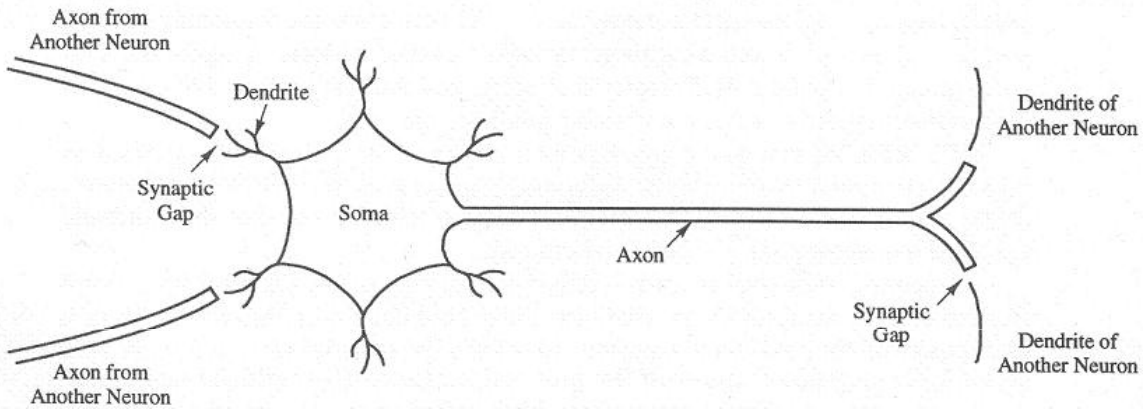
- O neurônio biológico e o neurônio artificial.

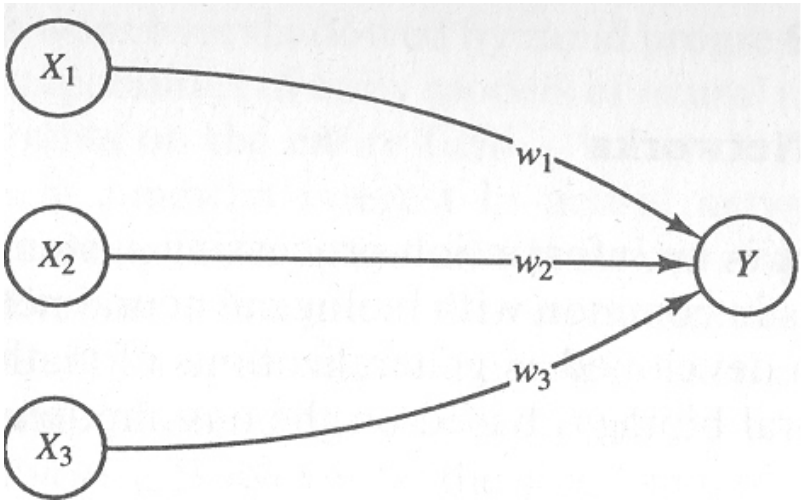


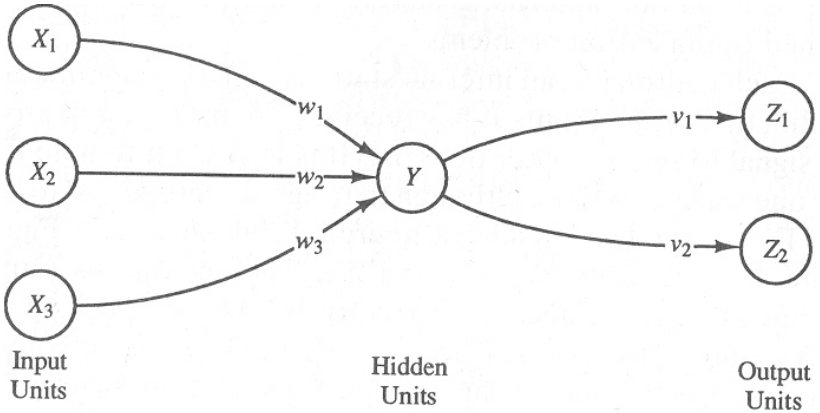


Preliminares

- O neurônio biológico e o neurônio artificial.







Perceptron Simples



Introdução

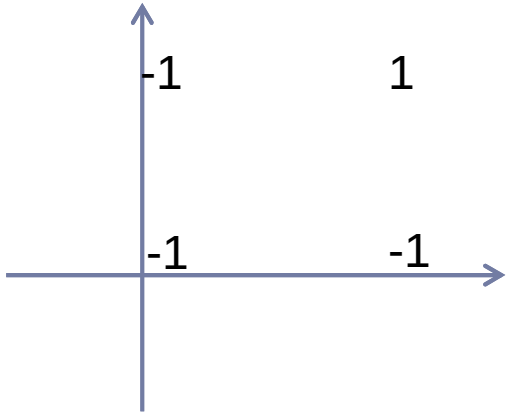
- Frank Rosenblatt introduziu e desenvolveu a classe de redes neurais artificiais chamada *Perceptrons*.

- Para nossos fins, considere que a meta da rede é classificar cada padrão de entrada como pertencente, ou não pertencente, a uma classe particular.
  - A pertinência é representada pela unidade de saída emitindo uma resposta +1;
  - A não pertinência é representada pela unidade de saída emitindo uma resposta -1;

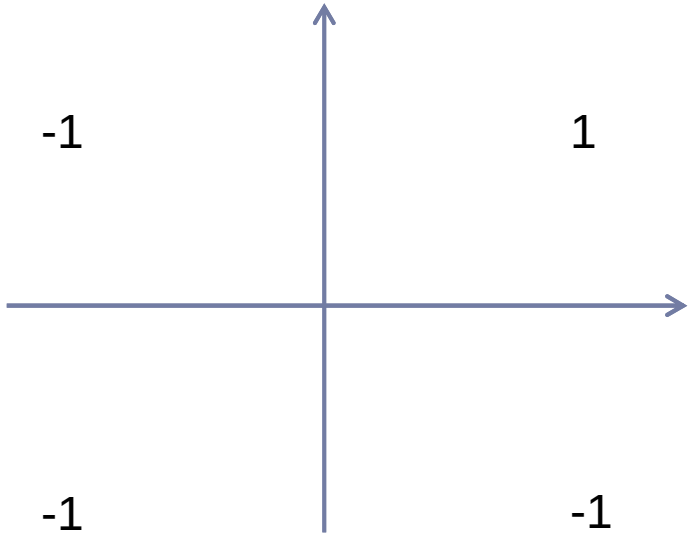
1, 1 → 1	1, 1 → 1
1, 0 → -1	1,-1 → -1
0, 1 → -1	-1, 1 → -1
0, 0 → -1	-1,-1 → -1
Entradas binárias e saídas bipolares	Entradas bipolares e saídas bipolares



Representando o problema de diferentes formas

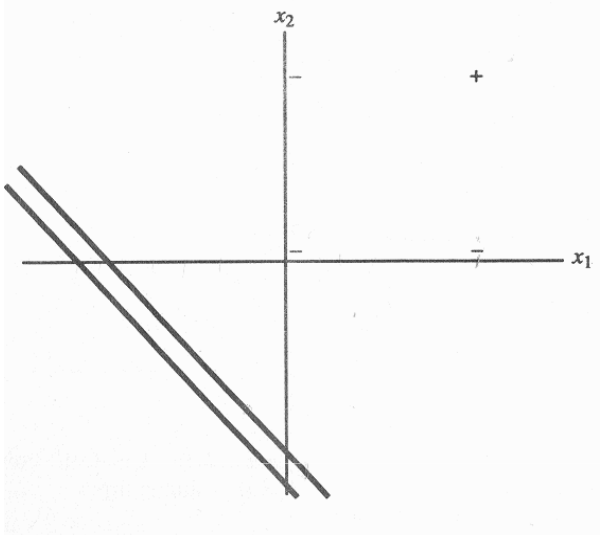


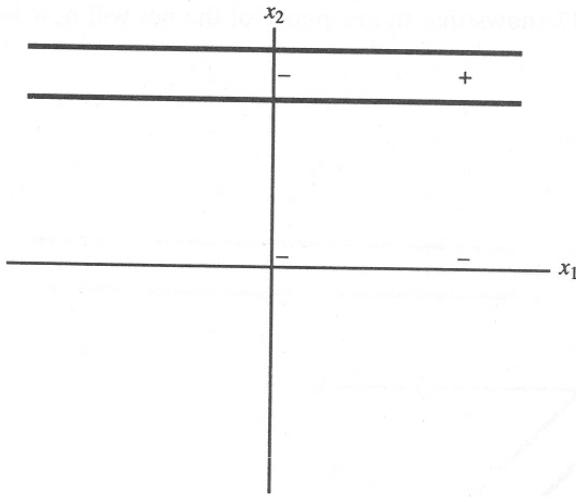
1, 1 → 1
1, 0 → -1
0, 1 → -1
0, 0 → -1

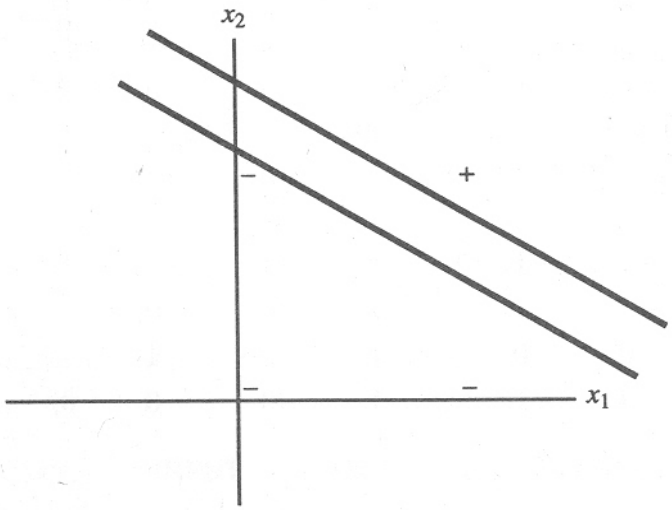


1, 1 → 1
1,-1 → -1
-1, 1 → -1
-1,-1 → -1

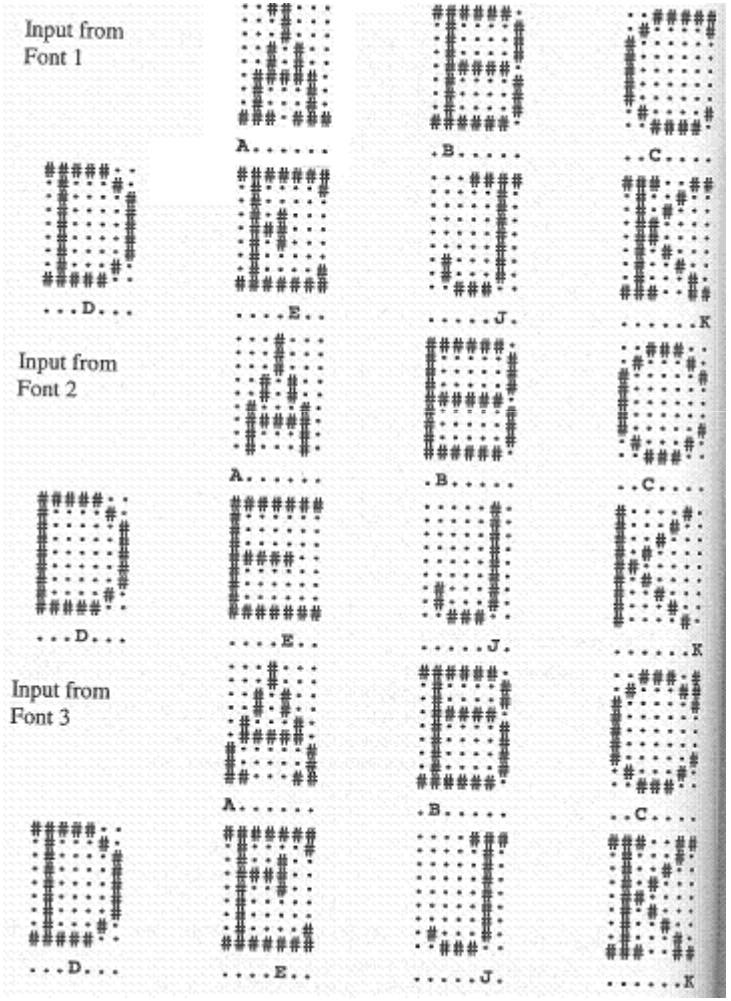
Separabilidade Linear



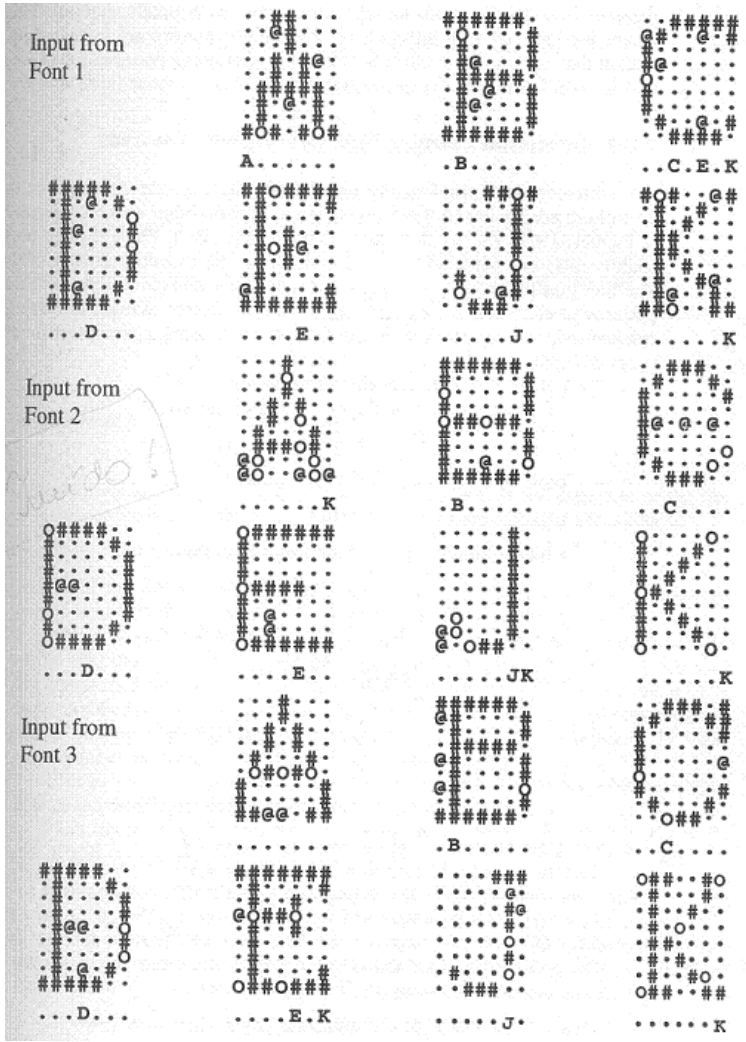




Resolvendo um problema – reconhecimento de caracteres



E agora?

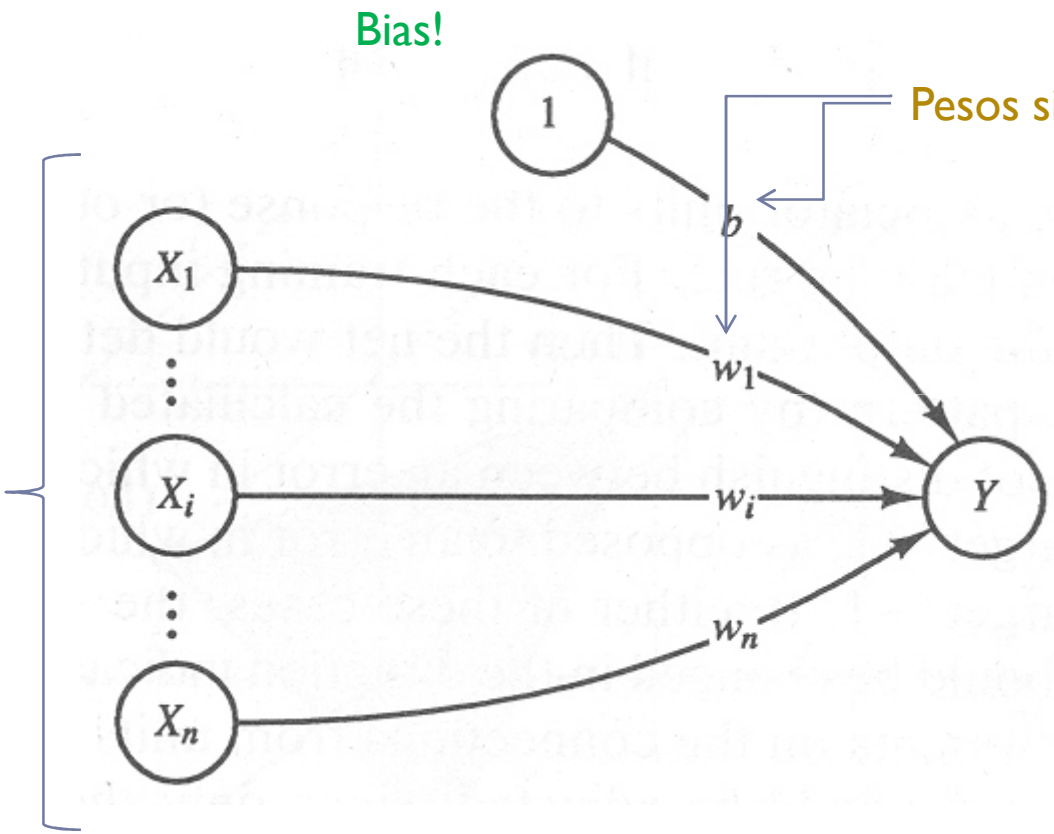


Primeiro: precisamos definir nossa rede Perceptron definindo sua **arquitetura**

Nossa “rede” será composta, por enquanto, por um único neurônio – o neurônio artificial (clássico)

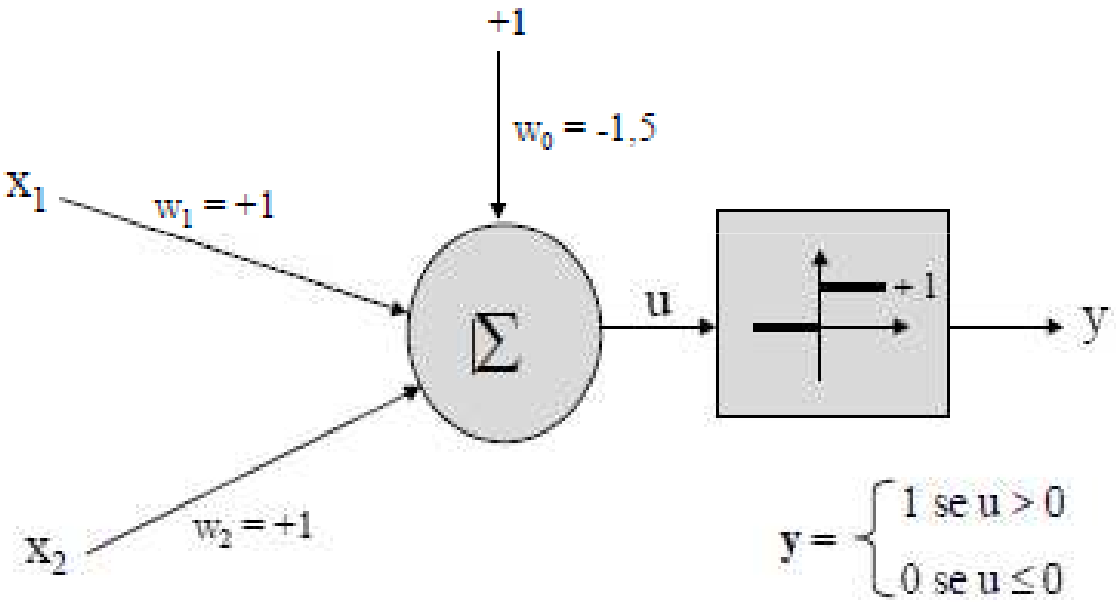


Arquitetura



Obs.: Aqui não estamos usando o neurônio McCulloc-Pits !!

Outra representação (uma instância)





Segundo: Precisamos implementar um algoritmo de aprendizado (treinamento) para que a rede APRENDA.

Mas, como assim?
Treinamento?
Ocorre aprendizado?





Algoritmo para treinamento

Passo 0:

Inicialize os pesos e o bias

(Por simplicidade, determine os pesos e bias como 0 (zero))

Determine a taxa de aprendizado α ($0 < \alpha <= 1$)

(Por simplicidade, α pode iniciar em 1).

Determine o limiar de disparo dos neurônios (θ)

Passo 1:

Enquanto a condição de parada é falsa, execute os passos 2-6



Algoritmo para treinamento

Passo 2:

dado de entrada : classificação conhecida

Para cada par de treinamento *s:t*, execute os passos 3-5.

Passo 3:

Determine as ativações das unidades de entrada: $x_i = s_i$

Passo 4:

Compute as respostas da unidade de saída:

$$y_in = b + \sum_i x_i w_i$$

$$y = f(y_in) = \begin{cases} 1 & if\ y_in > \theta \\ 0 & if\ -\theta \le y_in \le \theta \\ -1 & if\ y_in < -\theta \end{cases}$$



Algoritmo para treinamento

Passo 5:

Altere os pesos e bias se um erro ocorreu para o padrão em análise.

If y <> t,

$w_i(new) = w_i(old) + \alpha t x_i$
$b(new) = b(old) + \alpha t$

else

$w_i(new) = w_i(old)$
$b(new) = b(old)$

Passo 6:

Teste a condição de parada. Por exemplo, se nenhum peso mudou no passo 2 pare, senão, continue.





OK, AGORA FAÇA EXATAMENTE O QUE EU MANDO !!!!!

Algoritmo para treinamento (estudando melhor)

Passo 5:

Altere os pesos e bias se um erro ocorreu para o padrão em análise.

If y <> t,

$w_i(new) = w_i(old) + \alpha t x_i$

$b(new) = b(old) + \alpha t$

else

$w_i(new) = w_i(old)$

$b(new) = b(old)$

Somente unidades que contribuíram para o erro tem seus pesos ajustados (no caso de entradas binárias, uma entrada = 0 não contribui para o erro).

Precisamos do sinal da entrada e do sinal da saída desejada para corrigir o peso.

Analise o caso do problema de classificação binária com representação binária: se temos um **erro** quando **x** é negativo e **t** é negativo, é porque ou os pesos são negativos, fazendo a entrada no neurônio ser positiva e suficientemente forte para gerar uma resposta positiva – que é o erro; ou o peso é positivo mas não é forte o suficiente para fazer a entrada do neurônio cair para levar a uma saída negativa.
Sendo x e t negativos, o produto deles é um número positivo que será somado aos pesos. Assim, o peso caminha na direção de se tornarem positivos.

Na próxima iteração, a chance da resposta ser negativa (e correta) é maior, pois o sinal de entrada negativo multiplicado por um peso positivo e maior resultará em uma entrada mais fraca no neurônio.

Ainda nesse caso, o peso do bias vai decrementar, contribuindo para enfraquecer o sinal e levar a uma resposta negativa, como desejado.



Separabilidade Linear (estudando melhor)

Observando a função de ativação ....

$$y = ax + b$$

- Note que existe uma linha que separa a região de respostas positivas da região de respostas 0:

resposta 1

$w_1x_1 + w_2x_2 + b > \Theta,$

$$x_2 > -\frac{w_1}{w_2}x_1 + \frac{-b+\theta}{w_2}$$

- E uma linha que separa a região de respostas 0 da região de respostas negativas:

resposta -1

$w_1x_1 + w_2x_2 + b < -\Theta,$

$$x_2 < -\frac{w_1}{w_2}x_1 + \frac{-b-\theta}{w_2}$$

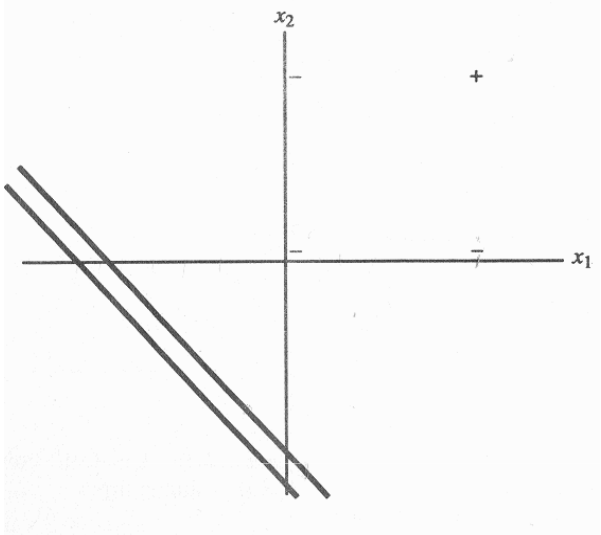
ativação > Θ resposta 1

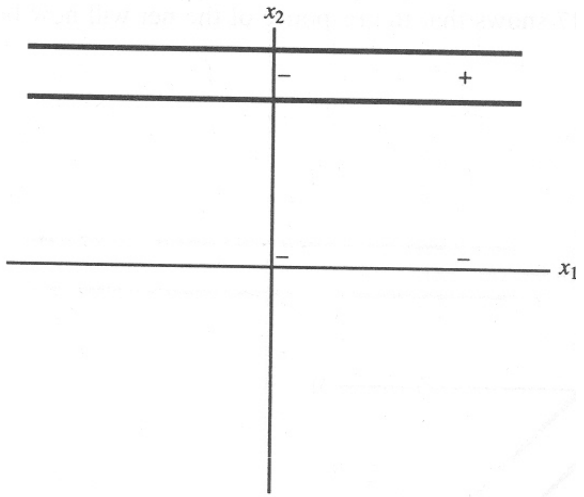
-Θ < ativação < Θ resposta 0

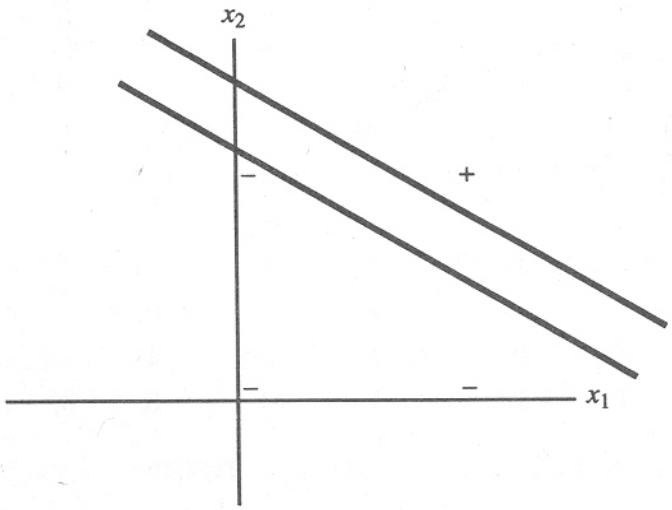
ativação < -Θ resposta -1

Se w1 ou w2 é igual a 0
a respectiva sinapse é nula e o termo
é retirado da função de cômputo
da entrada no neurônio.

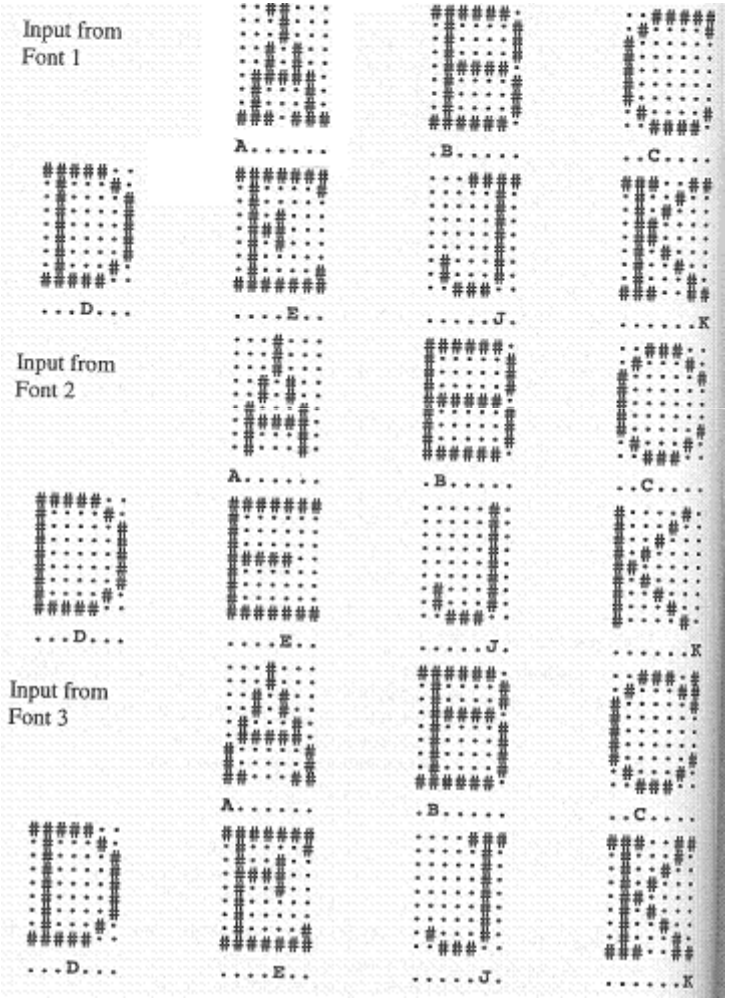
Separabilidade Linear (estudando melhor)

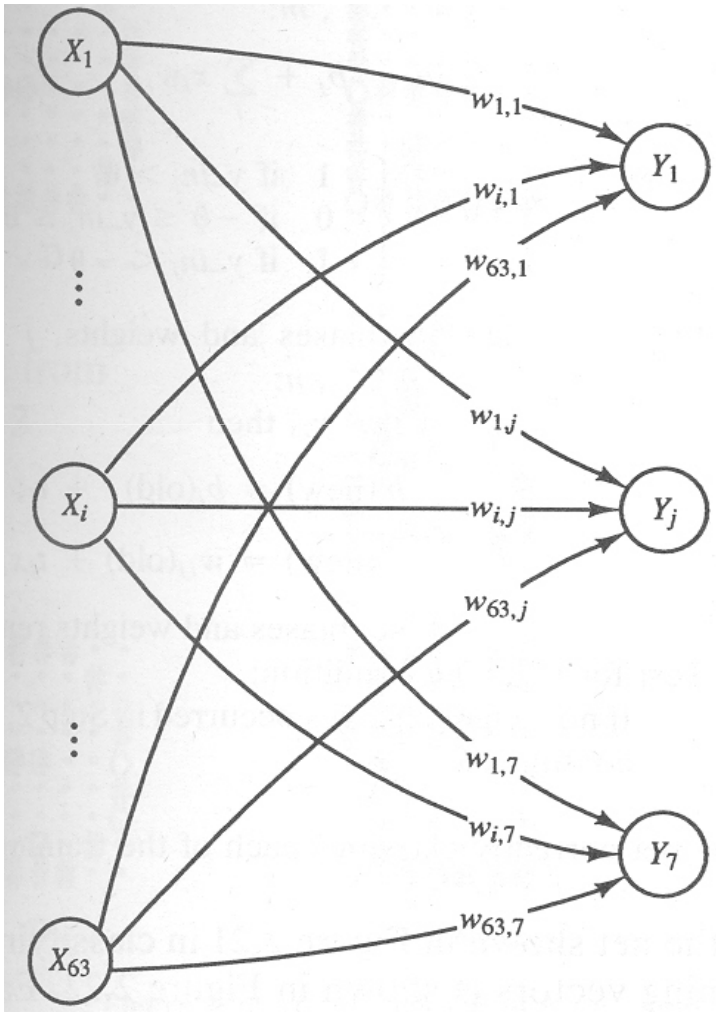




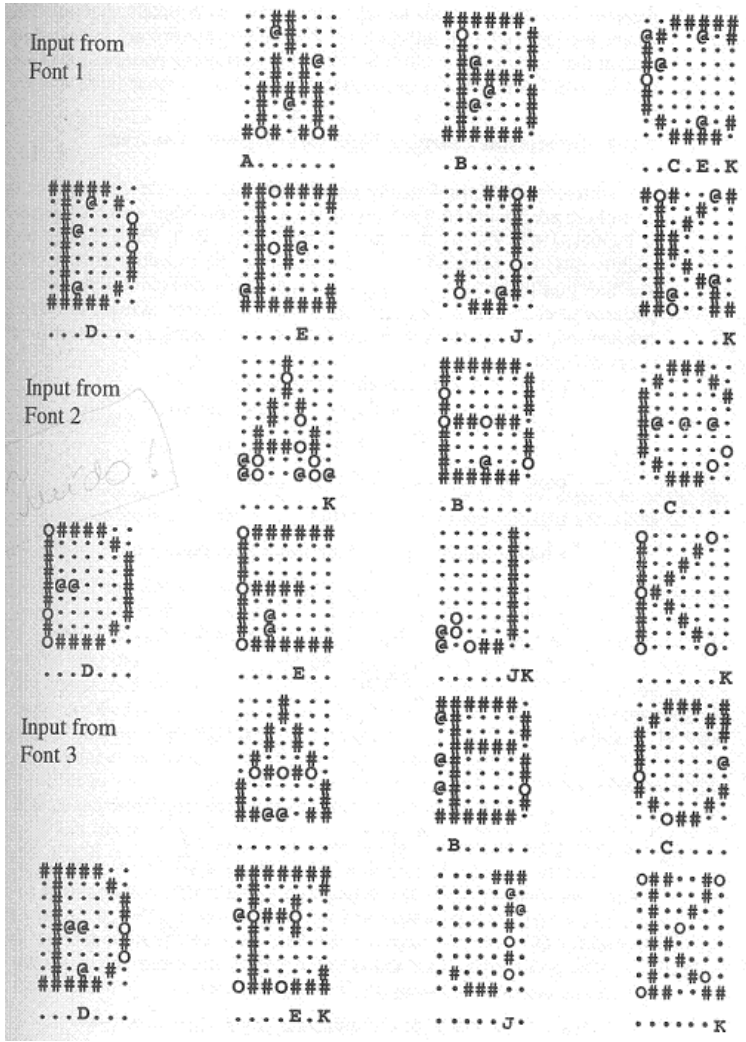


Resolvendo um problema – reconhecimento de caracteres





E agora?



Explorando o processo de aprendizado

- Considerando o problema do AND e:
  - taxa de aprendizado = 1;
  - pesos e bias iniciais = 0;
  - limiar de ativação = 0.2.

Entradas			y_in	y	Resposta esperada	Mudanças nos pesos			Pesos		
x_1	x_2	1							w_1	w_2	b
									0	0	0
1	1	1	0	0	1	1	1	1	1	1	1

- $w_1x_1 + w_2x_2 + b = 0*1 + 0*1 + 1*0 \rightarrow y_in = 0$
  - Θ: $-0.2 <= 0 < 0.2 \rightarrow y = 0$
    - resposta errada

plotando as retas →

- w1(new) = 0 + 1*1*1 = 1
- w2(new) = 0 + 1*1*1 = 1
  - b(new) = 0 + 1*1 = 1

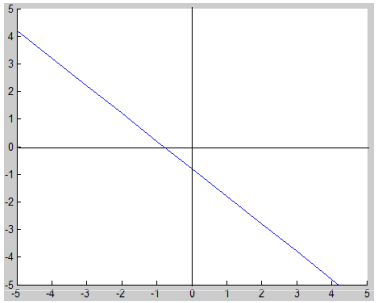
Explorando o processo de aprendizado

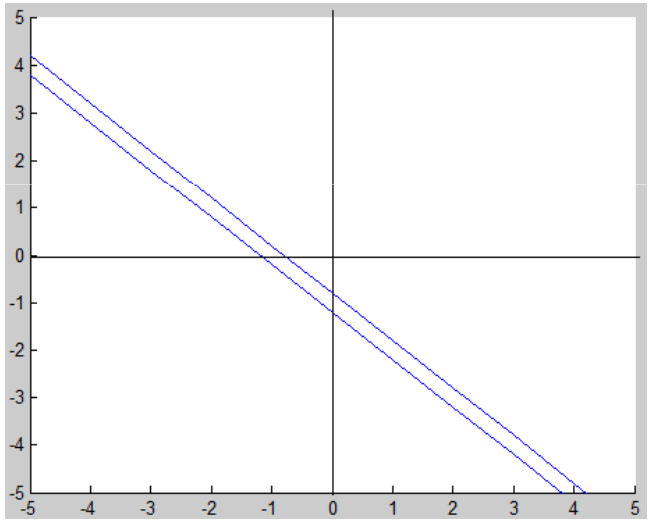
$$x_2 > -\frac{w_1}{w_2} x_1 + \frac{-b+\theta}{w_2}$$

y = ax + b

a = -w1/w2 = -1/1 = -1
b = -*b* + 0.2 = -1 + 0.2 = -0.8

y = - x - 0.8





$$x_2 < -\frac{w_1}{w_2} x_1 + \frac{-b-\theta}{w_2}$$

y = ax + b

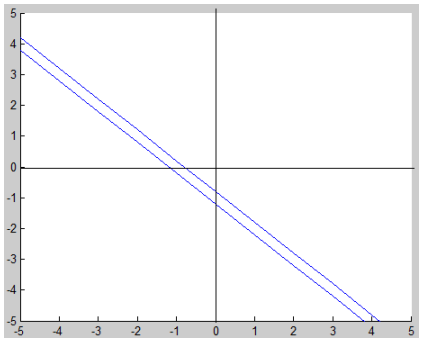
a = -w1/w2 = -1/1 = -1
b = -*b* - 0.2 = -1 – 0.2 = -1.2

y = - x – 1.2





Explorando o processo de aprendizado



Entradas			y_in	y	Resposta esperada	Mudanças nos pesos			Pesos		
x_1	x_2	1							w_1	w_2	b
									0	0	0
1	1	1	0	0	1	1	1	1	1	1	1
									1	1	1
1	0	1	2	1	-1	-1	0	-1	0	1	0

resposta errada

w1(new) = 1 + 1*1*-1 = 0
w2(new) = 1 + 1*0*-1 = 1
b(new) = 1 + 1*-1 = 0

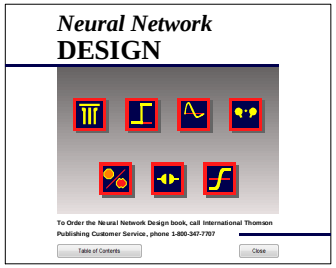
$$x_2 > -\frac{w_1}{w_2} x_1 - \frac{b+\theta}{w_2}$$

$$x_2 < -\frac{w_1}{w_2} x_1 - \frac{b-\theta}{w_2}$$

- Observe que é preciso mudar os pesos para mudar a reta de lugar.
- A resposta esperada era negativa e obtivemos uma resposta positiva, logo temos um erro de sinal na resposta.
- As retas estão abaixo da região onde deveriam estar e precisam ser reposicionadas mais acima. Precisam cortar o eixo x2 mais acima. Logo, o termo b da equação da reta precisa subir.
- Os pesos serão mais baixos (menores) e **b**, da equação da reta, será (-0 + 0.2) / 1 = + 0.2 (mais alto que - 0.8) e (-0 – 0.2) / 1 = -0.2 (mais alto que – 1.2).
- O sinal das entradas e dos pesos também influenciam na inclinação da reta. Se um dos pesos se tornar negativo, por exemplo, a inclinação da reta muda de direção.

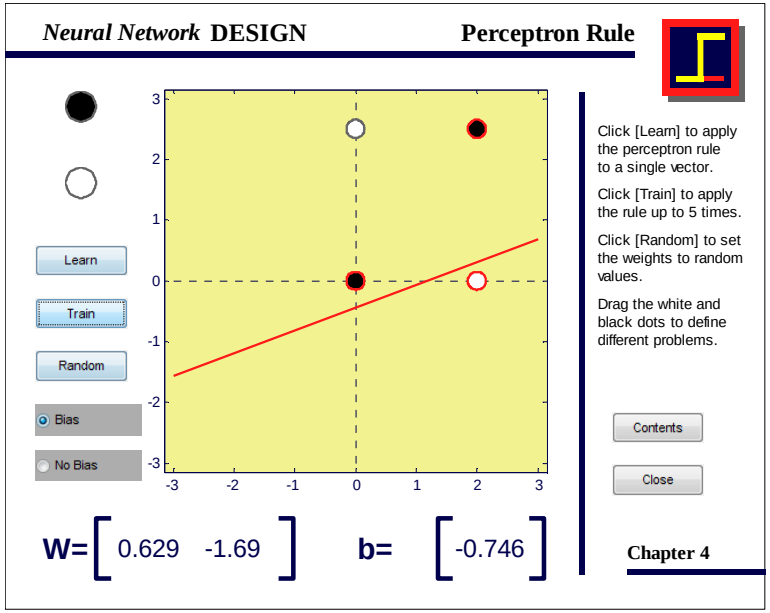
Estudando o comportamento ....

Simulações do Matlab



- nnd: carrega interface para buscar diferentes simulações.

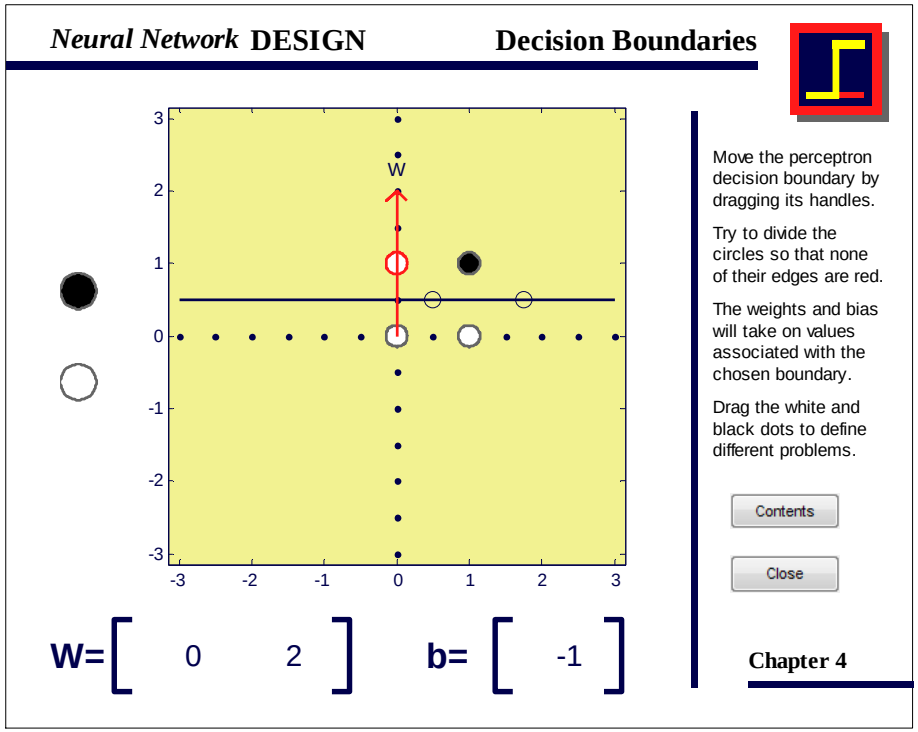
- nnd4db : mostra a plotagem da reta de divisão do espaço e os valores dos pesos e bias correlatos.





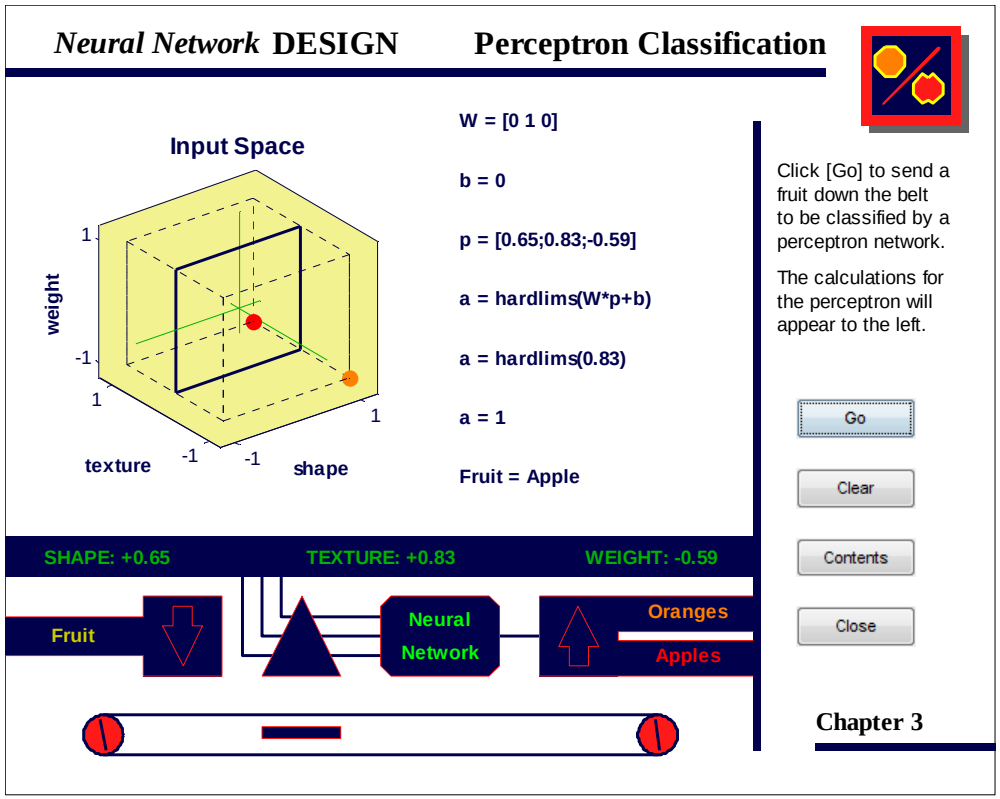
Estudando o comportamento ....

- Simulações do Matlab
  - nnd4pr: mostra a plotagem da reta de divisão do espaço e os valores dos pesos e bias correlatos, durante o processo de treinamento.



Estudando o comportamento ....

- Simulações do Matlab
  - nnd3pc: classificador de maças e laranjas (espaço de características tri-dimensional)



Informativo

Neurônio Artificial

- Neurônio artificial de McCulloch-Pitts
  - Talvez o neurônio artificial mais antigo (o primeiro a ser criado), de 1943.

- O neurônio artificial "clássico"



Informativo

Como funciona um neurônio artificial?

O neurônio de McCulloc-Pitts possui os seguintes requisitos (6):

1. A ativação do neurônio McCulloc-Pitts é binária. Isto é, em um determinado momento, ou o neurônio dispara (tem a ativação = 1) ou o neurônio não dispara (tem a ativação = 0).



Informativo

Como funciona um neurônio artificial?

O neurônio de McCulloc-Pitts possui os seguintes requisitos (6):

1. A ativação do neurônio McCulloc-Pitts é binária. Isto é, em um determinado momento, ou o neurônio dispara (tem a ativação = 1) ou o neurônio não dispara (tem a ativação = 0).

2. Esses neurônios são conectados por caminhos direcionados e pesados.



Informativo

Como funciona um neurônio artificial?

O neurônio de McCulloc-Pitts possui os seguintes requisitos (6):

1. A ativação do neurônio McCulloc-Pitts é binária. Isto é, em um determinado momento, ou o neurônio dispara (tem a ativação = 1) ou o neurônio não dispara (tem a ativação = 0).

2. Esses neurônios são conectados por caminhos direcionados e pesados.

3. O caminho de conexão é excitatório se o peso no caminho é positivo; caso contrário é inibitório. Todas as conexões excitatórias chegando em um neurônio tem os mesmos pesos.



Informativo

Como funciona um neurônio artificial?

O neurônio de McCulloc-Pitts possui os seguintes requisitos (6):

1. A ativação do neurônio McCulloc-Pitts é binária. Isto é, em um determinado momento, ou o neurônio dispara (tem a ativação = 1) ou o neurônio não dispara (tem a ativação = 0).

2. Esses neurônios são conectados por caminhos direcionados e pesados.

3. O caminho de conexão é excitatório se o peso no caminho é positivo; caso contrário é inibitório. Todas as conexões excitatórias chegando em um neurônio tem os mesmos pesos.

4. Cada neurônio tem um limiar fixo (threshold) tal que se a entrada do neurônio é maior que o limiar, então o neurônio dispara.



Informativo

Como funciona um neurônio artificial?

O neurônio de McCulloc-Pitts possui os seguintes requisitos (6):

1. A ativação do neurônio McCulloc-Pitts é binária. Isto é, em um determinado momento, ou o neurônio dispara (tem a ativação = 1) ou o neurônio não dispara (tem a ativação = 0).

2. Esses neurônios são conectados por caminhos direcionados e pesados.

3. O caminho de conexão é excitatório se o peso no caminho é positivo; caso contrário é inibitório. Todas as conexões excitatórias chegando em um neurônio tem os mesmos pesos.

4. Cada neurônio tem um limiar fixo (threshold) tal que se a entrada do neurônio é maior que o limiar, então o neurônio dispara.

5. O limiar é determinado tal que uma inibição é absoluta. Isto é, qualquer entrada inibitória não-zero impedirá o neurônio de disparar.



Informativo

Como funciona um neurônio artificial?

O neurônio de McCulloc-Pitts possui os seguintes requisitos (6):

1. A ativação do neurônio McCulloc-Pitts é binária. Isto é, em um determinado momento, ou o neurônio dispara (tem a ativação = 1) ou o neurônio não dispara (tem a ativação = 0).

2. Esses neurônios são conectados por caminhos direcionados e pesados.

3. O caminho de conexão é excitatório se o peso no caminho é positivo; caso contrário é inibitório. Todas as conexões excitatórias chegando em um neurônio tem os mesmos pesos.

4. Cada neurônio tem um limiar fixo (threshold) tal que se a entrada do neurônio é maior que o limiar, então o neurônio dispara.

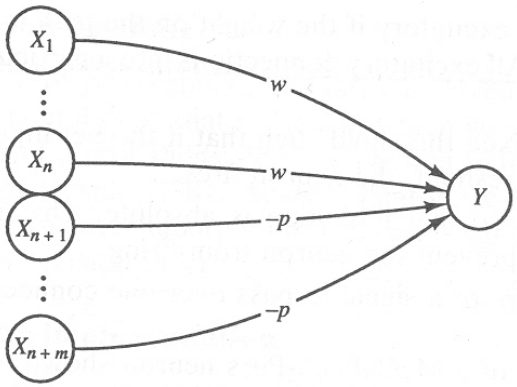
5. O limiar é determinado tal que uma inibição é absoluta. Isto é, qualquer entrada inibitória não-zero impedirá o neurônio de disparar.

6. É necessário um "passo de tempo" para um sinal passar por um *link* de conexão.



Informativo

O neurônio de McCulloc-Pitts



Temos que determinar o limiar de disparo do neurônio Y e então determinar os pesos das conexões, com o intuito de resolver um problema.

Um neurônio McCulloc-Pitts Y deve receber sinais de qualquer número de outros neurônios.

$$y_in = \sum_{i=1}^{n+m} x_i w(ou\ p)_{ij}$$

Cada *link* de conexão é ou excitatório (w, com w > 0) ou inibitório (-p, com p > 0).

Na figura temos **n** unidades $X_1, \ldots X_n$, as quais enviam sinais excitatórios para a unidade Y, e **m** unidades, $X_{n+1}, \ldots X_{n+m}$, as quais enviam sinais inibitórios.

A função de ativação para a unidade Y é: $f(y_in) = \begin{cases} 1 & if\ y_in \geq \theta \\ 0 & if\ y_in < \theta \end{cases}$ onde y_in é o sinal total recebido e θ é o limiar.

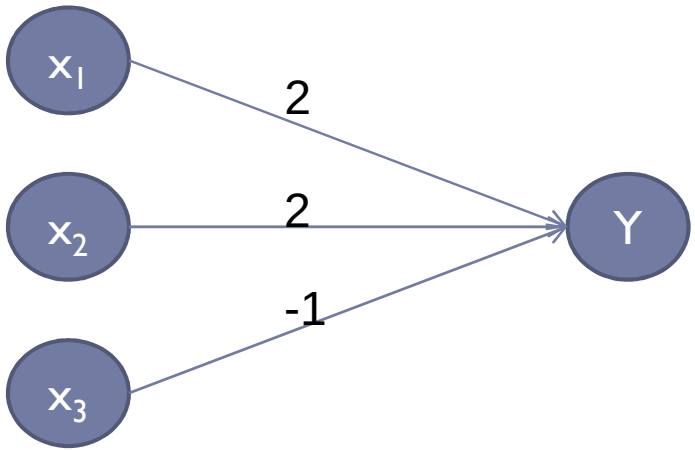
Y dispara se recebe **k** ou mais entradas excitatórias e nenhuma inibitória, onde $kw >= \theta > (k-1)w$.

Embora todos os pesos excitatórios que entram em uma unidade (Y1) devam ser os mesmos, os pesos que entram em um outra unidade (Y2) não precisam ter o mesmo valor dos que estão entrando na primeira unidade (Y1).

Informativo

O neurônio de McCulloc-Pitts

Exemplo:



- As conexões excitatórias tem o mesmo peso (já que estão entrando na mesma unidade).
- O *threshold* para a unidade Y é 4: para os valores de pesos mostrados, este é o único valor de *threshold* que permitirá que Y dispare algumas vezes, mas o impedirá de disparar se ele receber um sinal diferente de 0 na conexão inibitória.



Informativo

Implementando funções lógicas

- Exemplo: implementação de portas lógicas

- Função AND:
  - A função AND resulta na resposta "true" se ambas as entradas são valoradas com "true"; caso contrário a resposta é "false".
  - Se nós representamos "true" por '1' e "false" por '0', isto nos dá o seguinte conjunto de 4 pares (padrões) :

1,1 → 1

1,0 → 0

0,1 → 0

0,0 → 0

Entradas binárias e saídas binárias



Informativo

Resultado

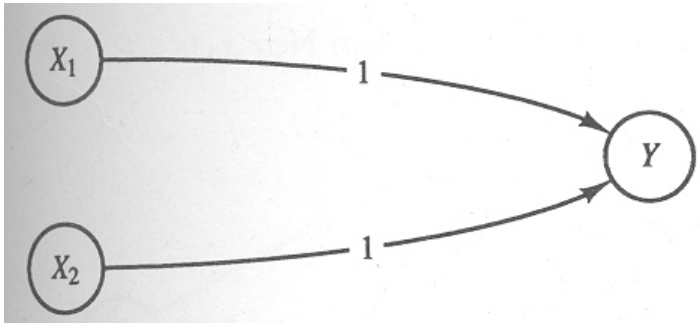


Figure 1.14 A McCulloch-Pitts neuron to perform the logical AND function.

Threshold em Y é 2.

Threshold em Y é 2.

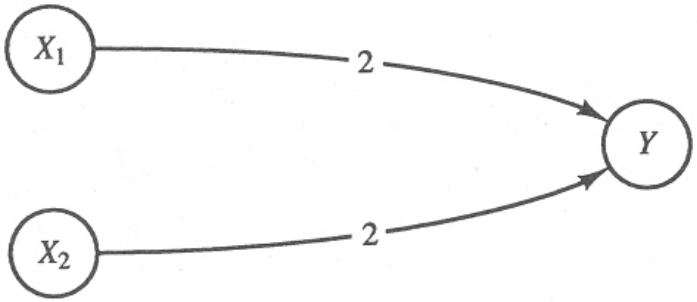


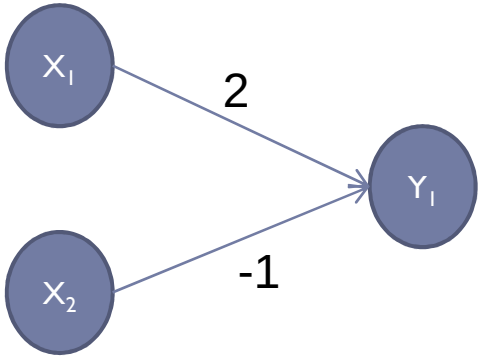
Figure 1.15 A McCulloch-Pitts neuron to perform the logical OR function.

Informativo

Veja a modelagem para o AND NOT e para o XOR

AND NOT

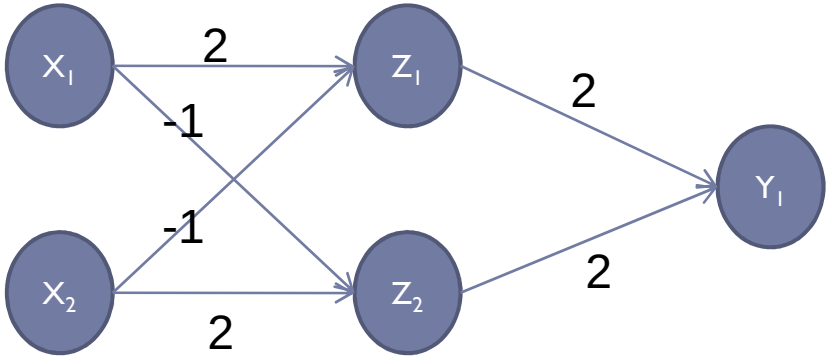
limiar = 2 em Y_1



x_1	x_2	Y
1	1	0
1	0	1
0	1	0
0	0	0

XOR

limiar = 2 em Z_1, Z_2 e Y_1



x_1	x_2	Y
1	1	0
1	0	1
0	1	1
0	0	0

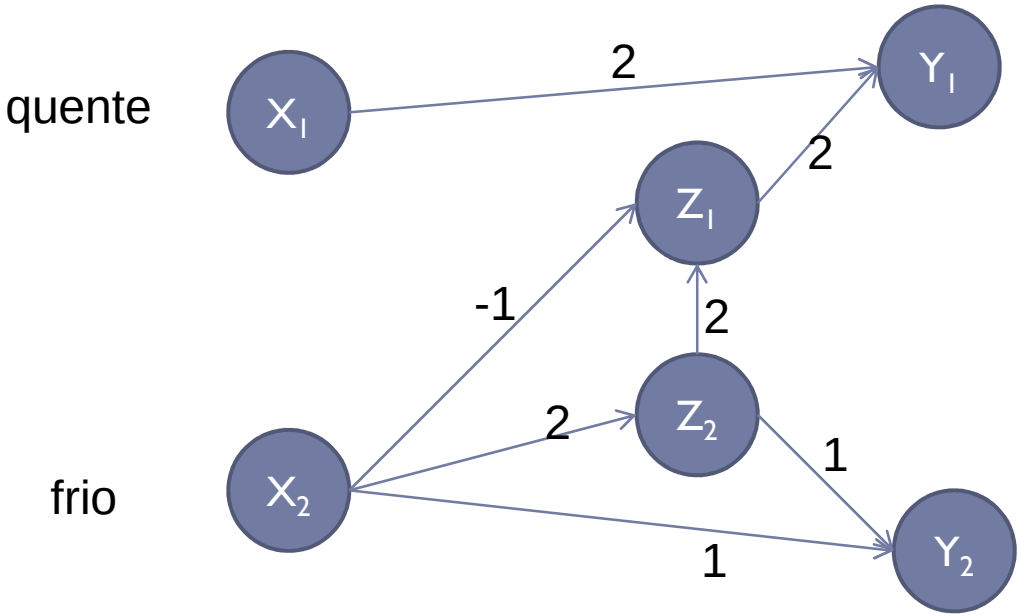
x_1 XOR $x_2 \leftrightarrow (x_1$ AND NOT $x_2)$ OR $(x_2$ AND NOT $x_1)$

Informativo

Como resolvido no livro da Fausett

Mais um exemplo ...

- Modelando o fenômeno que se um estímulo **frio** é aplicado à pele de uma pessoa por um período de tempo muito pequeno, a pessoal perceberá **quente**. Contudo, se o mesmo estímulo é aplicado por um período mais longo, a pessoa perceberá **frio**.
- O modelo é projetado apenas para a primeira percepção de quente ou frio recebida pelas unidades perceptoras.
- Neurônios X_1 e X_2 representam os receptores de quente e frio, respectivamente. Os neurônios Y_1 e Y_2 são os perceptores.



Todos os neurônios tem o threshold = 2.

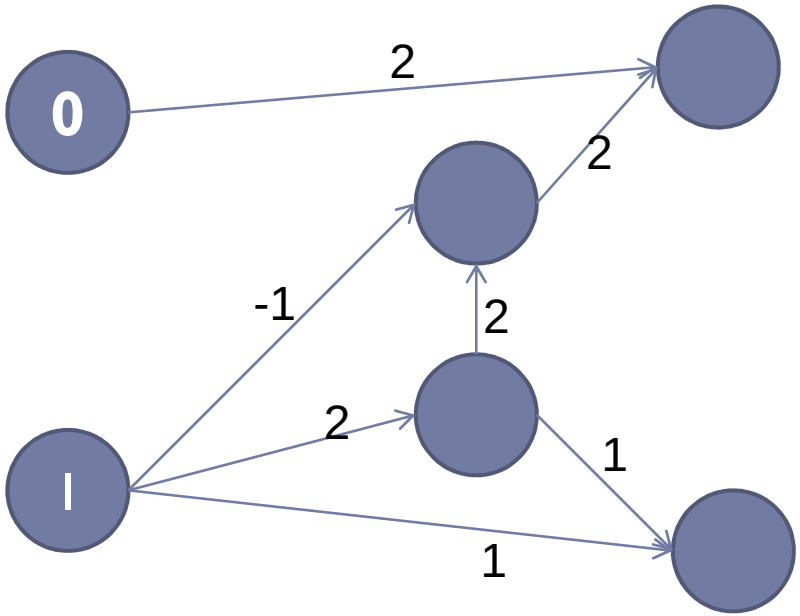
A entrada do sistema será (1,0) se **quente** é aplicado, e (0,1) se **frio** é aplicado.

A resposta desejada do sistema é que **frio** seja percebido se o estímulo **frio** é aplicado por dois passos de tempo.

Teste 1: o estímulo de frio é aplicado por um passo de tempo.

Como resolvido no livro da Fausett

- O estímulo **frio** é aplicado e então retirado (nada é então aplicado).



Tempo = 0 **(t = 0)**

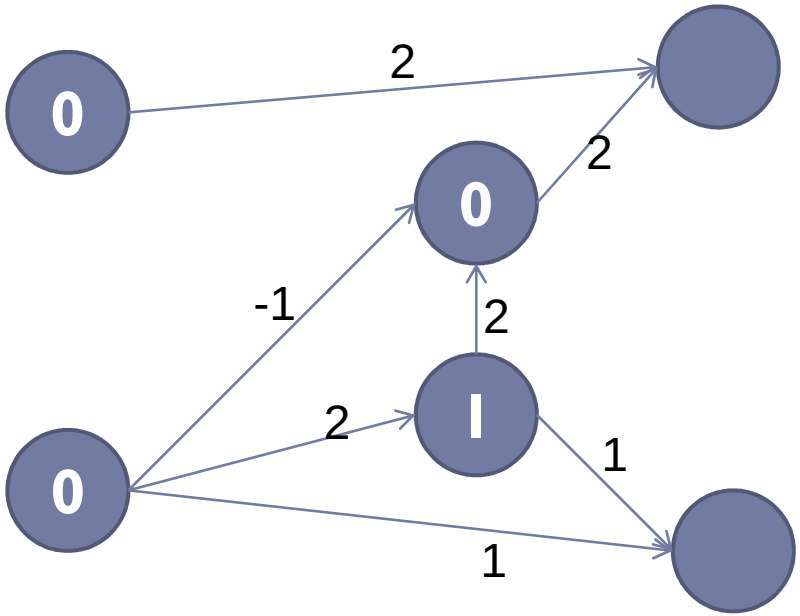
As ativações conhecidas são as ativações dos neurônios receptores.

Informativo

Teste 1: o estímulo de frio é aplicado por um passo de tempo.

Como resolvido no livro da Fausett

- O estímulo **frio** é aplicado e então retirado (nada é então aplicado).



Tempo = 1 **(t = 1)**

As ativações dos neurônios receptores são todas 0 (não há estímulo **frio** ou **quente**).

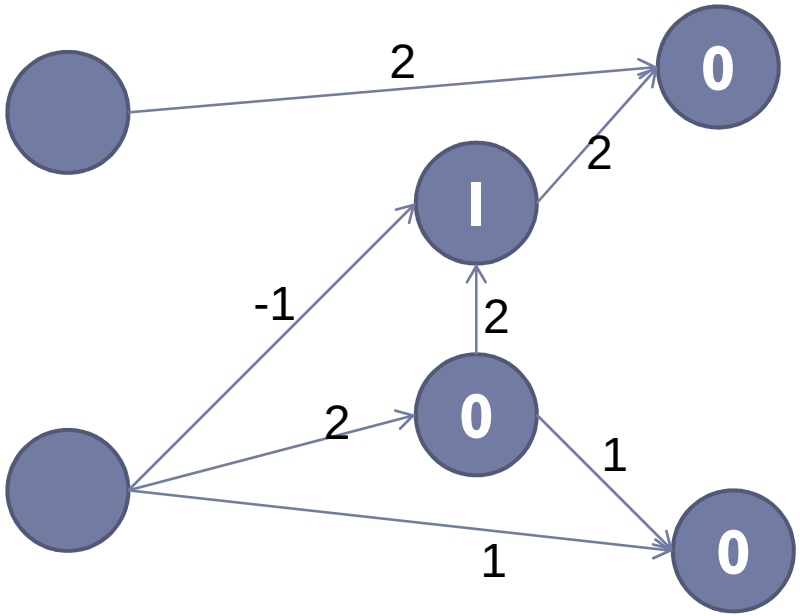
As ativações nas unidades Z são baseadas nas ativações das unidades X no t = 0.

Informativo

Teste 1: o estímulo de frio é aplicado por um passo de tempo.

Como resolvido no livro da Fausett

- O estímulo **frio** é aplicado e então retirado (nada é então aplicado).



Tempo = 2 **(t = 2)**

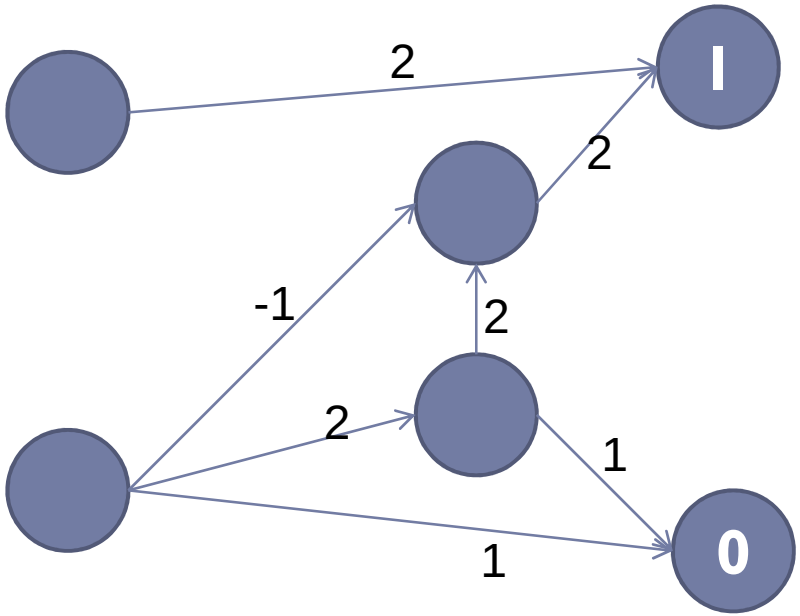
As ativações dos receptores não são mostradas porque seus valores em t = 2 não determinar a primeira resposta da rede.

Informativo

Teste 1: o estímulo de frio é aplicado por um passo de tempo.

Como resolvido no livro da Fausett

- O estímulo **frio** é aplicado e então retirado (nada é então aplicado).



Tempo = 3 **(t = 3)**

A percepção é quente, visto que Y_1 tem uma ativação 1 e Y_2 tem uma ativação 0.

Informativo

Exercícios

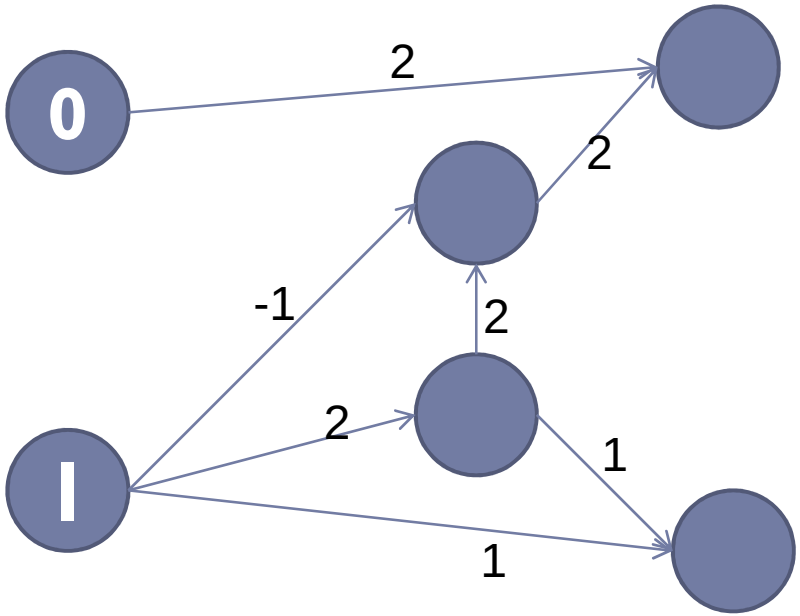
- Repita o procedimento para:
  - Teste 2: um estímulo frio é aplicado em dois passos de tempo.
  - Teste 3: um estímulo quente é aplicado em um passo de tempo.
- Estude como as portas lógicas foram organizadas na rede neural proposta (observe os passos de tempo).



Teste 2: um estímulo frio é aplicado em dois passos de tempo.

Como resolvido no livro da Fausett

- O estímulo **frio** é aplicado duas vezes seguidas.

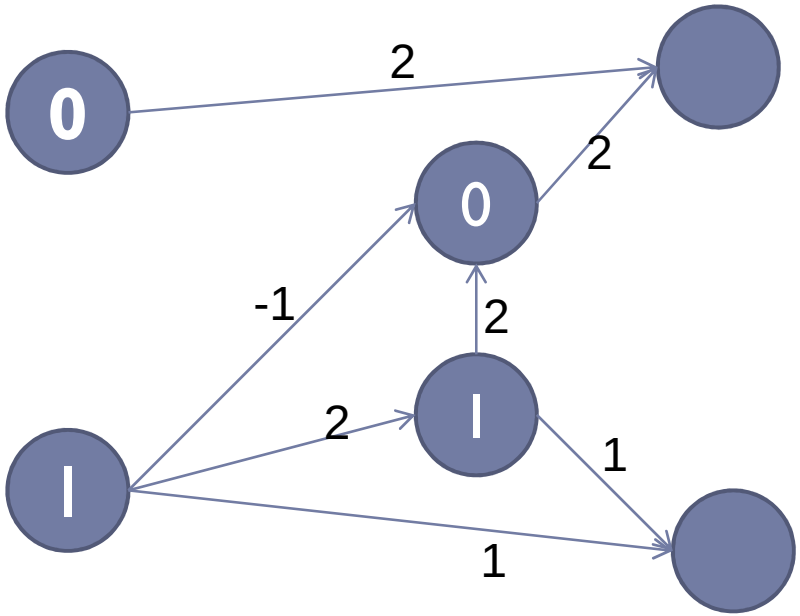


Tempo = 0 **(t = 0)**

Teste 2: um estímulo frio é aplicado em dois passos de tempo.

Como resolvido no livro da Fausett

- O estímulo **frio** é aplicado duas vezes seguidas.



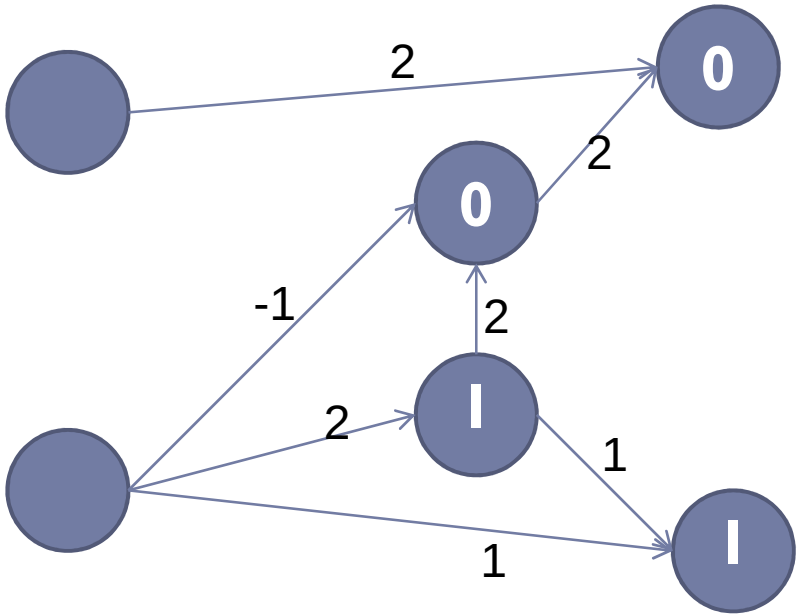
Tempo = 1 **(t = 1)**

Informativo

Teste 2: um estímulo frio é aplicado em dois passos de tempo.

Como resolvido no livro da Fausett

- O estímulo **frio** é aplicado duas vezes seguidas.



Tempo = 2 **(t = 2)**

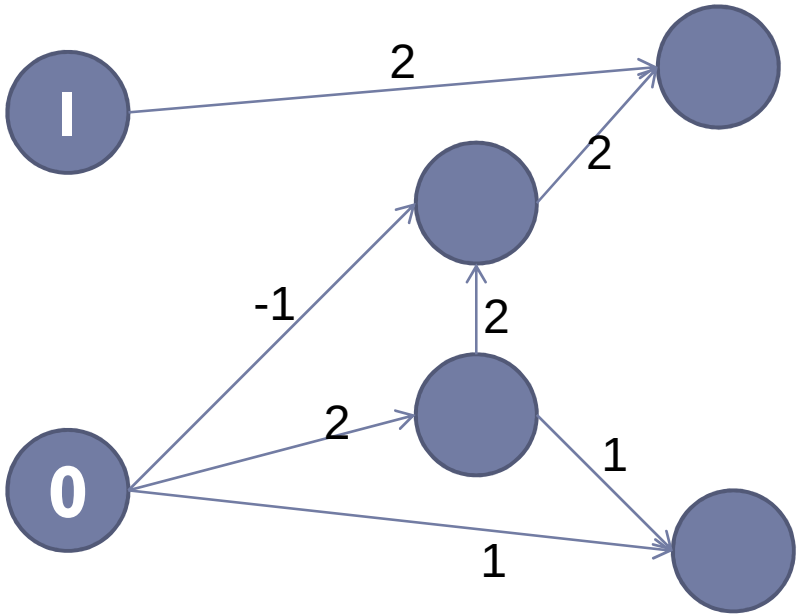
As ativações das unidades de entrada não são especificadas porque a primeira resposta da rede para o estímulo frio, aplicado duas vezes, não deve ser influenciada pelo fato do estímulo ser removido depois dos dois passos.

Embora as respostas das unidades Z sejam mostradas, as respostas das saídas são determinadas pelas ativações de todas as unidades no t = 1.

Informativo

Teste 3: um estímulo quente é aplicado em um passo de tempo.

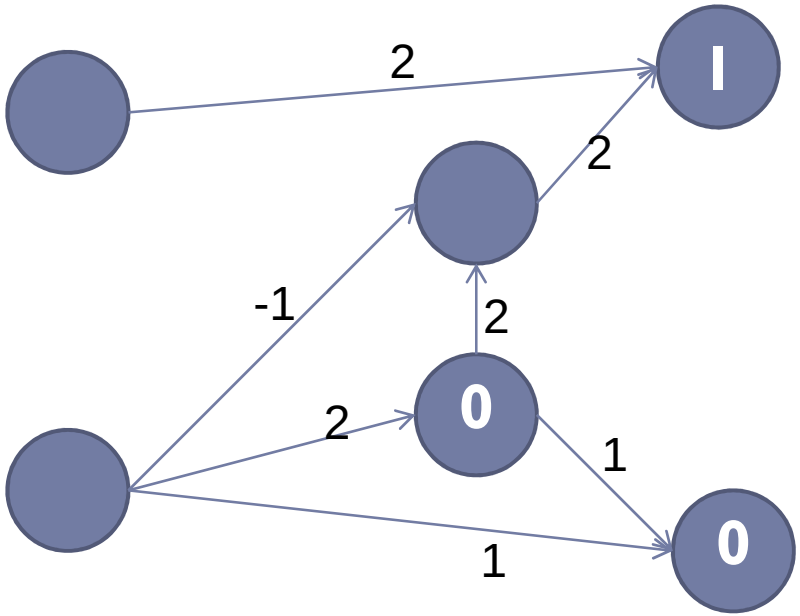
- O estímulo **quente** é aplicado por um passo de tempo.



Tempo = 0 **(t = 0)**

Teste 3: um estímulo quente é aplicado em um passo de tempo.

- O estímulo **quente** é aplicado por um passo de tempo.



Tempo = 1 **(t = 1)**

A unidade Y1 dispara porque ela recebeu um sinal de X1.

Y2 não dispara porque ela requer sinais tanto de X2 quanto de Z2 para disparar, e X2 tem uma ativação de 0 no t = 0.

Observações referentes aos questionamentos em aula

- Sobre o modelo não funcionar para a apresentação da seguinte sequência de estímulos:
  - 0 1 (frio) / 0 1 (frio) / 0 0 (retirada de estímulo)
  - Fornecendo a resposta de percepção de quente (que nos parece não fazer sentido, embora considerando o raciocínio de dois em dois passos de tempo, a resposta faria sentido).

- A seguinte observação é colocada na Fausett (complementei o slide 20 para não causar novamente o entendimento errado da capacidade do modelo), seguindo a apresentação original do problema em artigo de McCulloch e Pitts:
  - The model is designed to give only the first perception of heat or cold that is received by the perceptor units.



Informativo

Observações referentes aos questionamentos em aula

- Em relação aos "passos de tempo" e "guardar o sinal por mais de um passo de tempo".
  - O projeto deste modelo está baseado no uso (combinação e sequenciamento) de portas lógicas.
  - A ativação de um neurônio não persiste por mais do que "um passo de tempo", ou seja, ela não é "guardada".



Informativo

Observações referentes aos questionamentos em aula

- O disparo ou não de um neurônio de processamento está diretamente ligado às condições para isso. Assim:
  - O neurônio z1, no tempo = 1, no teste 1, dispara sem necessitar da saída do neurônio z2, por conta da regra de inibição total, que neste caso permite a implementação da porta lógica AND NOT, onde a entrada 1 no neurônio X com peso sináptico inibitório leva à resposta 0 (veja tabela verdade e modelagem do AND NOT no slide 19.
  - O neurônio z1, no tempo = 1, no teste 3, não tem sua ativação determinada, pois a entrada 0 no neurônio X, seguindo a mesma lógica de observar a tabela verdade da porta lógica AND NOT, não é determinante, e é preciso a informação da segunda entrada do neurônio z1 (neste caso, a saída do neurônio z2).
  - O neurônio Y1, no tempo = 1, no teste 3, tem sua ativação determinada diretamente do estímulo no tempo = 0 no neurônio X1 apenas, visto que na porta lógica implementada por Y1, basta uma das entradas serem 1 para que ele tenha condições de disparo.



Informativo

Observações referentes aos questionamentos em aula

- Sobre considerar que a ausência de informação de disparo num neurônio é equivalente a dizer que ele não dispara (e portanto, neste caso, seria admitir que ele está com o sinal 0)
  - Considerando o modelo de nosso exemplo, não encontrei problemas, dado que a ativação é binária.
  - Para uso de outros tipos de ativação (bipolar ou contínua – que não é o caso usado no modelo de neurônio McCulloc-Pitts) é necessário uma análise específica.



Informativo

Adaline



Introdução

- ADALINE – Adaptive Linear Neuron
- Proposta por Widrow e Hoff, 1960.
- Usa uma regra de aprendizado (de ajuste de peso) baseada na minimização do erro.
- A função de ativação é a função identidade.
- A arquitetura é igual à do Perceptron simples.



Algoritmo para treinamento

Passo 0:

Inicialize os pesos e o bias

(Por simplicidade, determine os pesos e bias como valores randômicos e pequenos)

Determine a taxa de aprendizado α ($0 < \alpha <= 1$)

(Por simplicidade, α pode iniciar em 0,1).

Passo 1:

Enquanto a condição de parada é falsa, execute os passos 2-6



Algoritmo para treinamento

Passo 2:

Para cada par de treinamento bipolar *s:t*, execute os passos 3-5.

Passo 3:

Determine as ativações das unidades de entrada: $x_i = s_i$

Passo 4:

Compute as respostas da unidade de saída:

$$y_in = b + \sum_i x_i w_{ij}$$

Passo 5:

Altere todos pesos e bias

$$w_i(new) = w_i(old) + \alpha\,(t - y_in)x_i$$

$$b(new) = b(old) + \alpha\,(t - y_in)$$

Lembre-se que a função de ativação no treinamento é a função identidade.

Para a aplicação a função step pode ser usada e um limiar deve ser estabelecido.

Passo 6:

Teste a condição de parada. Se a maior alteração de pesos ocorrida no passo 2 é menor do que uma tolerância pré-especificada, então pare, senão continue.



Exercícios

- Faça o teste de mesa do treinamento do Perceptron para o problema do AND e do OR, usando diferentes formas de representação:
  - Entradas binárias e saídas binárias (neste caso, use uma função de ativação binária)
  - Entradas binárias e saídas bipolares (neste caso, use uma função de ativação bipolar)
  - Entradas bipolares e saídas bipolares (neste caso, use uma função de ativação bipolar)

- Faça testes de mesa para a Adaline.

- Implemente os algoritmos de treinamento das redes Perceptron e Adaline. Faça sua implementação de maneira genérica (ou seja, aceitando diferentes números de neurônios de entrada, diferentes números de neurônios de saída e diferentes funções de ativação)



Multilayer Perceptron (MLP)



Resumo

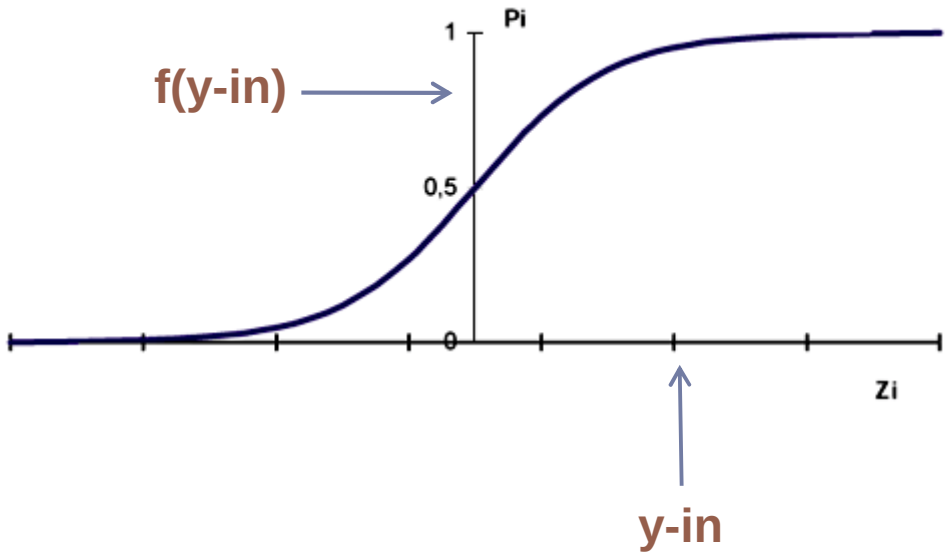
- Tipicamente é composto de:
  - um conjunto de neurônios sensoriais (ou nós fonte) que constitui a camada de entrada da rede;
  - uma ou mais camadas escondidas de neurônios que fazem processamento de sinal;
  - a camada de saída da rede.

  - Supervisionada e comumente treinada com o algoritimo de aprendizado supervisionado *backpropagation* (retro propagação do erro).
    - algoritmo baseado na regra de aprendizado de correção de erro;



Características

- O modelo de cada neurônio na rede possui uma função de ativação não linear (diferenciável em todos os seus pontos)
  - comumente é usada a função logística

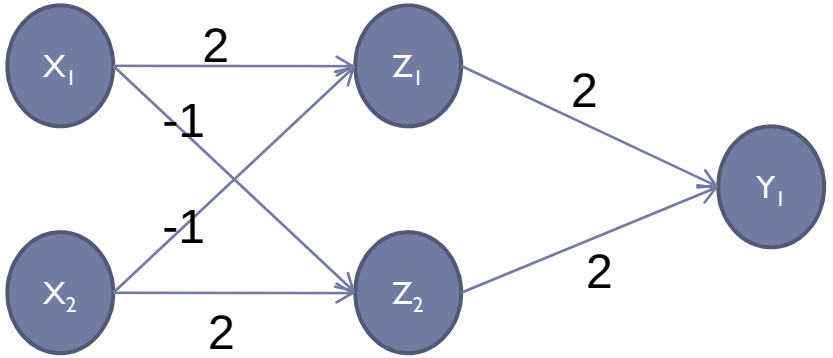


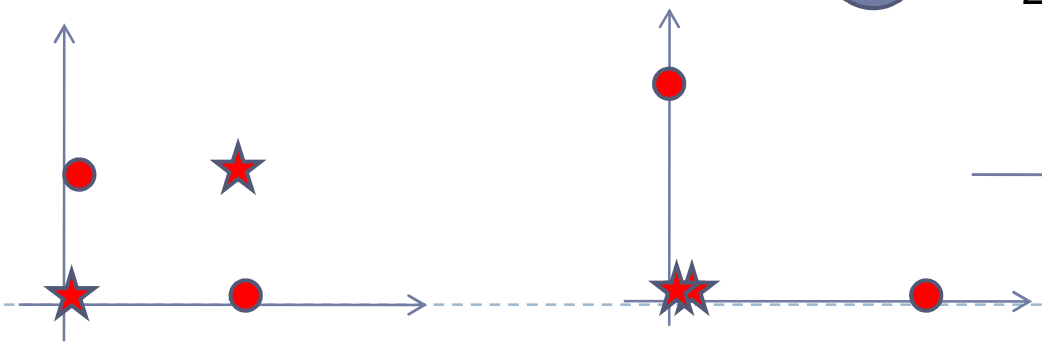
Características

- As camadas escondidas extraem, progressivamente, informações dos padrões de entrada.
  - Veja o que acontece com os dados do problema XOR quando passam por um neurônio escondido:

x1	x2	z1	z2	Y
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

Essa rede é com neurônios McCulloc-Pitts, mas o raciocínio para uma rede com neurônios Perceptron é similar.





O problema do XOR foi transformado em um problema linearmente separável pela ação da camada escondida.

Backpropagation



Backpropagation

- Método de treinamento conhecido como "retropropagação do erro" ou "regra delta generalizada".
  - Atua como um minimizador do erro observado nas saídas de uma rede neural artificial.

- Atua sobre uma arquitetura de rede neural de várias camadas, *feedfoward* e pode ser usado para resolver problemas em diferentes áreas.



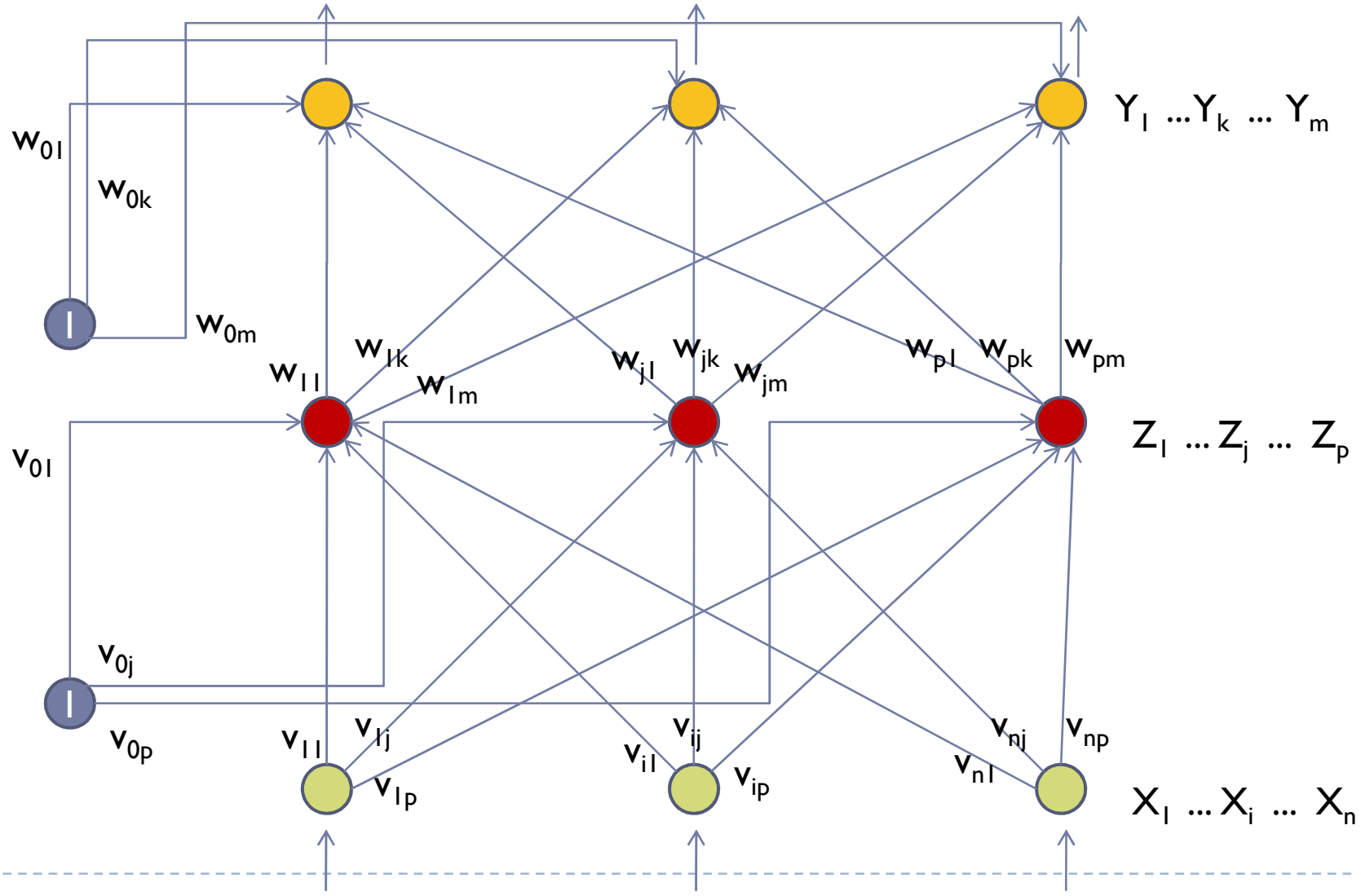
Backpropagation

- O treinamento envolve três estágios:
  - A passagem (feedforward) dos padrões de treinamento;
  - O cálculo e retropropagação do erro associado;
  - O ajuste de pesos.

- Os neurônios fazem dois tipos de computação:
  - a clássica – ativação do neurônio mediante entradas e uma função de ativação não-linear
  - o cálculo do sinal de erro – computação de um vetor gradiente



Arquitetura



Algoritmo para treinamento

Passo 0: Inicialize os pesos (valores randômicos pequenos)

Passo 1: Enquanto a condição de parada é falsa, execute os passos 2-9.

Passo 2: Para cada par de treinamento, execute os passos 3-8.

Feedforward (primeira fase)

Passo 3:

Passo 4: Próximos slides

Passo 5:

Backpropagation of error (segunda fase)

Passo 6:

Passo 7: Próximos slides

Update weights and biases (terceira fase)

Passo 8: Próximos slides

Passo 9: Teste a condição de parada



Algoritmo para treinamento - Feedforward (primeira fase)

Passo 3: Cada unidade de entrada (X_i, i = 1 ... n) recebe um sinal de entrada x_i e o dissipa para todas as unidades na camada acima (unidades escondidas).

Passo 4: Cada unidade escondida (Z_j, j = 1 ... p) soma suas entradas pesadas,

$$z_in_j = v_{0j} + \sum_{i=1}^{n} x_i v_{ij}$$

aplica sua função de ativação para computar seu sinal de saída,

$$z_j = f(z_in_j)$$

e envia o sinal para todas as unidades na camada acima (unidades de saída).

Passo 5: Cada unidade de saída (Y_k, k = 1 ... m) soma suas entradas pesadas

$$y_in_k = w_{0k} + \sum_{j=1}^{p} z_j w_{jk}$$

e aplica sua função de ativação para computar seu sinal de saída,

$$y_k = f(y_in_k)$$



Algoritmo para treinamento - Backpropagation of error (segunda fases)

Passo 6: Cada unidade de saída (Y_k, k = 1 ... m) recebe uma classificação correspondente ao padrão de entrada, computa seu termo de erro de informação

$$\delta_k = (t_k - y_k) f'(y_in_k)$$

calcula seu termo de correção de pesos

$$\Delta w_{jk} = \alpha \delta_k z_j$$

calcula seu termo de correção de bias

$$\Delta w_{0k} = \alpha \delta_k$$

e envia δ_k para as unidades cada camada abaixo.



Algoritmo para treinamento - Backpropagation of error (segunda fases)

Passo 7: Cada unidade de saída (Z_j, j = 1 ... p) soma suas entradas delta (vindas das unidades da camada acima)

$$\delta_in_j = \sum_{k=1}^{m} \delta_k w_{jk}$$

multiplica pela derivada de sua função de ativação para calcular seu termos de erro de informação

$$\delta_j = \delta_in_j \; f'(z_in_j)$$

calcula seu termo de correção de pesos

$$\Delta v_{ij} = \alpha \delta_j x_i$$

e calcula seu termo de correção de bias

$$\Delta v_{0j} = \alpha \delta_j$$



Algoritmo para treinamento - Update weights and biases (terceira fase)

Passo 8: Cada unidade de saída (Y_k, k = 1 ... m) altera seu bias e seus pesos (j = 0 ... p):

$$w_{jk}(new) = w_{jk}(old) + \Delta w_{jk}$$

Cada unidade escondida (Zj, j = 1 ... p) altera seu bias e seus pesos (i = 0 ... n):

$$v_{ij}(new) = v_{ij}(old) + \Delta v_{ij}$$



Procedimento de aplicação

Passo 0: Considere os pesos obtidos no algoritmo de treinamento

Passo 1: Para cada vetor de entrada, execute os Passos 2-4

Passo 2: Para i = 1 ... n: determine a ativação da unidade de entrada x_i

Passo 3: Para j = 1 ... p:

$$z_in_j = v_{0j} + \sum_{i=1}^{n} x_i v_{ij}$$

$$z_j = f(z_in_j)$$

Passo 4: Para k = 1 ... m:

$$y_in_k = w_{0k} + \sum_{j=1}^{p} z_j w_{jk}$$

$$y_k = f(y_in_k)$$



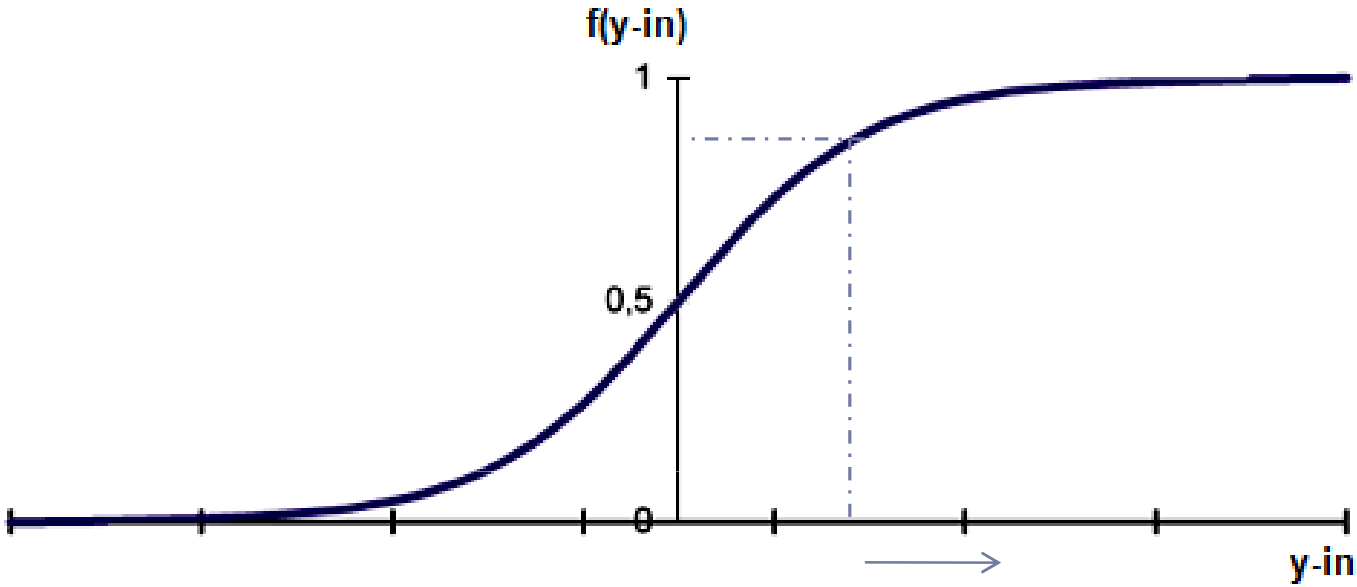
Funções de Ativação

- Exemplos

$f_1(x) = \frac{1}{1+\exp(-x)}$	$f_1'(x) = f_1(x)\,[1 - f_1(x)]$
$f_2(x) = \frac{2}{1+\exp(-x)} - 1$	$f_2'(x) = \frac{1}{2}[1 + f_2(x)][1 - f_2(x)]$



Discutindo

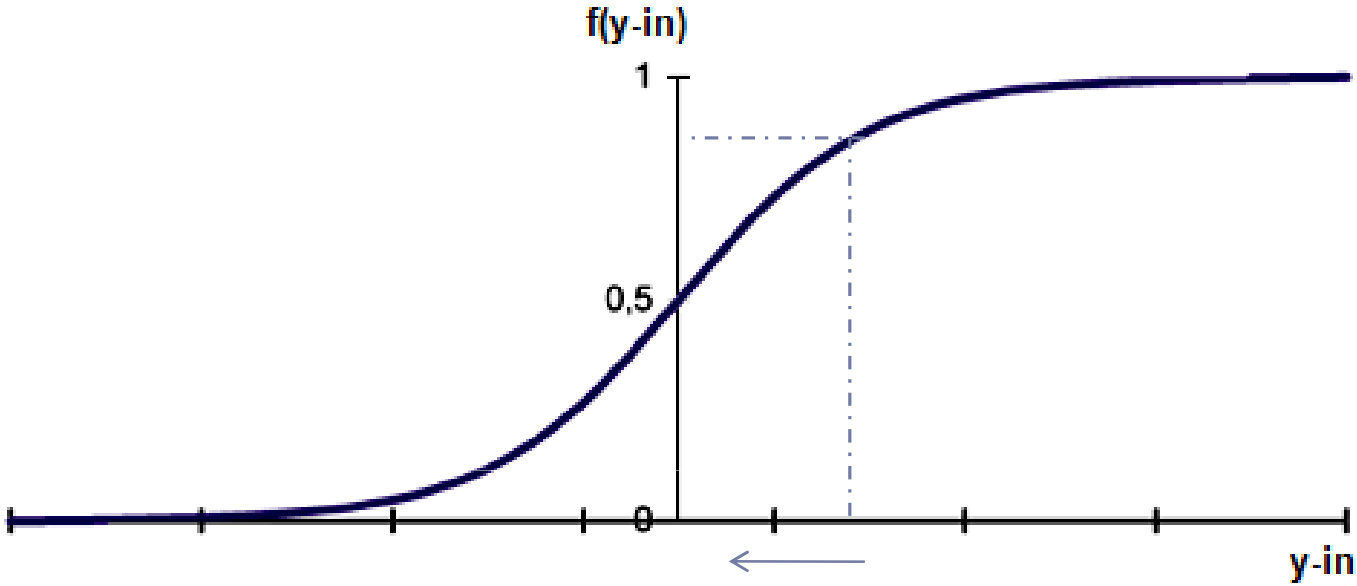


$x_{1..n}$	y-in	f(y-in)	Y(desejado)	Y (desejado) – f(y-in)	f'(y-in)
$x_{1..n}$	1,5	$1/(1+e^{-1,5})$ = **0,81**	**1**	1- 0,81 = 0,19 (positivo)	0,81*(1-0,81) = 0,14 (positivo)

O valor esperado é maior que o obtido. É preciso que a entrada do neurônio seja mais forte para que o valor da ativação do neurônio seja mais alto.

A derivada indica a direção de crescimento da função. Combinando o valor da derivada ao erro, tem-se um passo na direção de correção do erro.

Discutindo

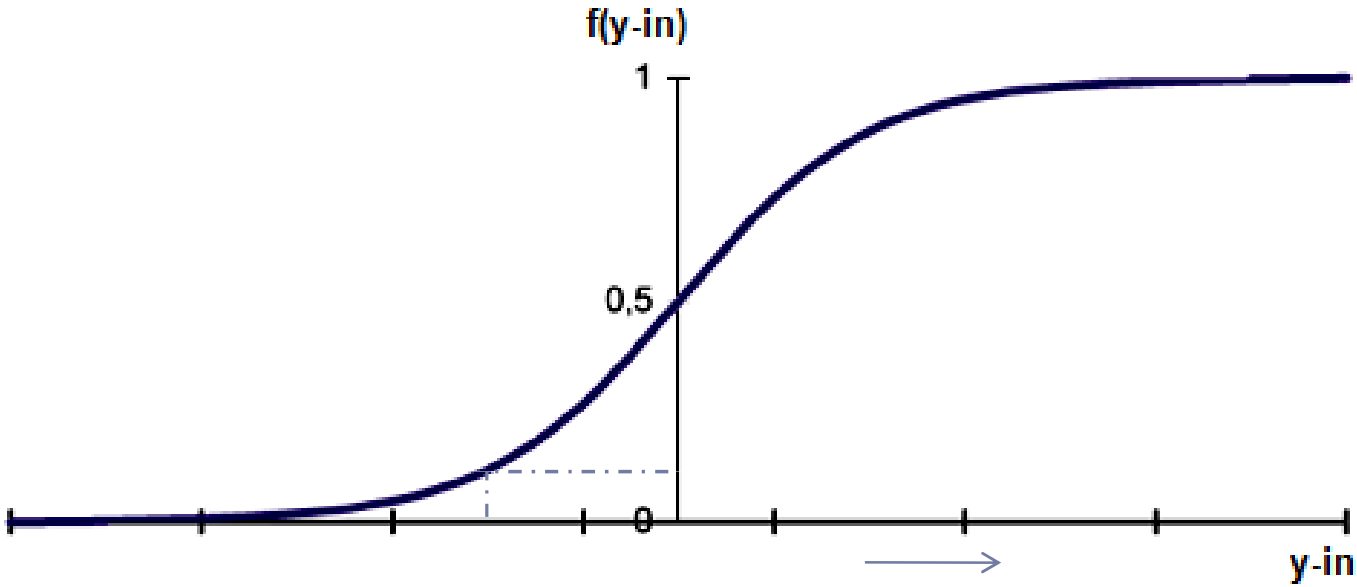


$x_{1..n}$	y-in	f(y-in)	Y(desejado)	Y (desejado) – f(y-in)	f'(y-in)
$x_{1..n}$	1,5	$1/(1+e^{-1,5})$ = **0,81**	**0**	0 - 0,81 = -0,81 (negativo)	0,81*(1-0,81) = 0,14 (positivo)

O valor esperado é menor que o obtido. É preciso que a entrada do neurônio seja mais fraca para que o valor da ativação do neurônio seja mais baixo.

A derivada indica a direção de crescimento da função. Combinando o valor da derivada ao erro, tem-se um passo na direção de correção do erro.

Discutindo

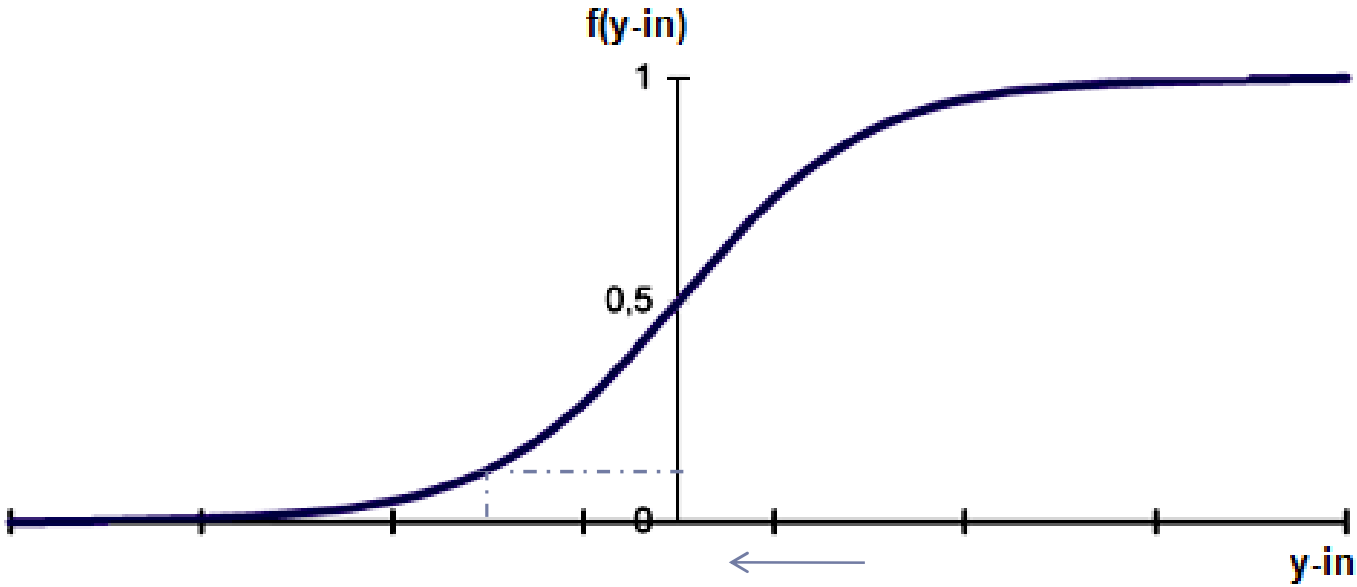


$x_{1..n}$	y-in	f(y-in)	Y(desejado)	Y (desejado) – f(y-in)	f'(y-in)
$x_{1..n}$	-1,5	$1/(1+e^{-(-1,5)})$ = **0,18**	**1**	1- 0,18 = **0,82** (positivo)	0,18*(1-0,18) = 0,14 (positivo)

O valor esperado é maior que o obtido. É preciso que a entrada do neurônio seja mais forte para que o valor da ativação do neurônio seja mais alto.

A derivada indica a direção de crescimento da função. Combinando o valor da derivada ao erro, tem-se um passo na direção de correção do erro.

Discutindo

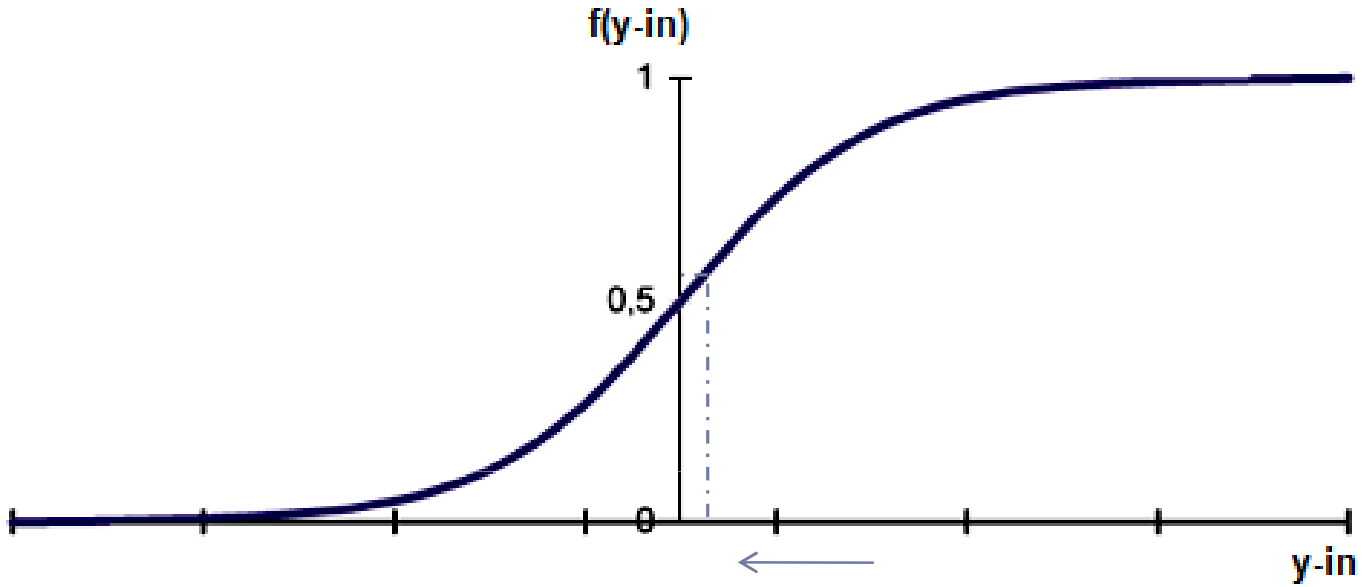


$x_{1..n}$	y-in	f(y-in)	Y(desejado)	Y (desejado) – f(y-in)	f'(y-in)
$x_{1..n}$	-1,5	$1/(1+e^{-(-1,5)})$ = **0,18**	**0**	0 – 0,18 = -0,18 (negativo)	0,18*(1-0,18) = 0,14 (positivo)

O valor esperado é menor que o obtido. É preciso que a entrada do neurônio seja mais fraca para que o valor da ativação do neurônio seja mais baixo.

A derivada indica a direção de crescimento da função. Combinando o valor da derivada ao erro, tem-se um passo na direção de correção do erro.

Discutindo

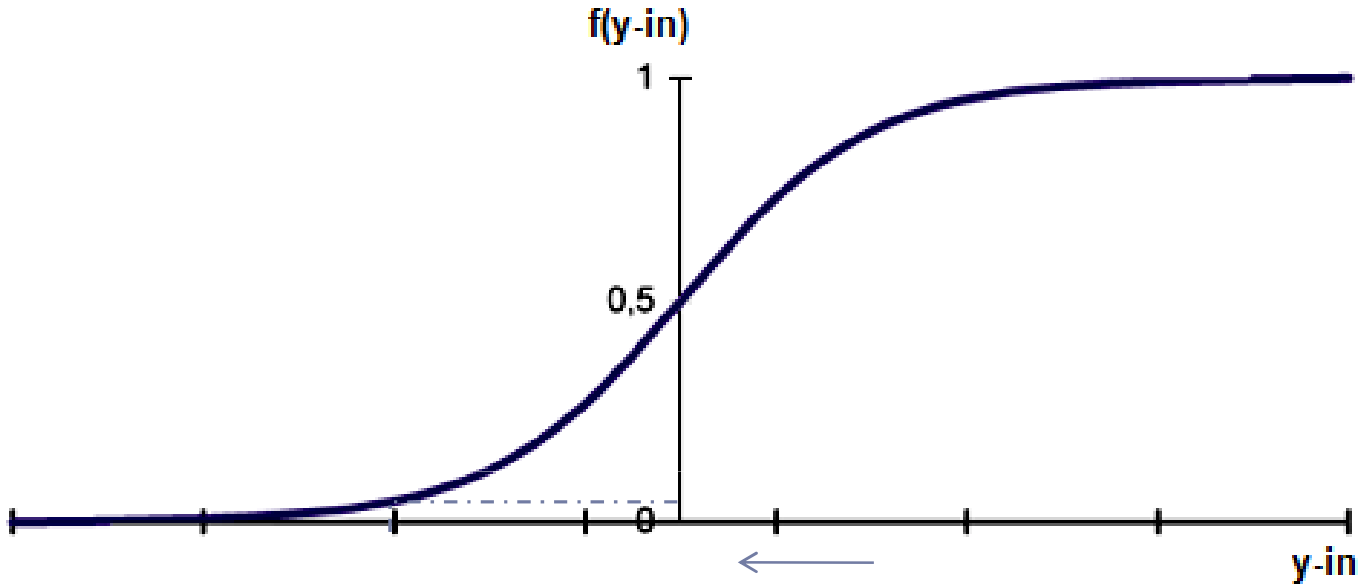


$x_{1..n}$	y-in	f(y-in)	Y(desejado)	Y (desejado) – f(y-in)	f'(y-in)
$x_{1..n}$	0,2	$1/(1+e^{-0,2})$ = **0,54**	**0**	0 – 0,54 = -0,54 (negativo)	0,54*(1-0,54) = 0,24 (positivo)

O valor esperado é menor que o obtido. É preciso que a entrada do neurônio seja mais fraca para que o valor da ativação do neurônio seja mais baixo.

A derivada indica a direção de crescimento da função. Combinando o valor da derivada ao erro, tem-se um passo na direção de correção do erro.

Discutindo



$x_{1..n}$	y-in	f(y-in)	Y(desejado)	Y (desejado) – f(y-in)	f'(y-in)
$x_{1..n}$	-2	$1/(1+e^{-(-2)})$ = **0,11**	**0**	0 – 0,11 = -0,11 (negativo)	0,11*(1-0,11) = 0,09 (positivo)

O valor esperado é menor que o obtido. É preciso que a entrada do neurônio seja mais fraca para que o valor da ativação do neurônio seja mais baixo.

A derivada indica a direção de crescimento da função. Combinando o valor da derivada ao erro, tem-se um passo na direção de correção do erro.